

2026
Software Requirements Specifications
Uni Textbook Marketplace

For **Project Day**

Presented by:

NexusDev

Software Requirements Specifications (SRS)

Project:	Uni Textbook Marketplace
Team:	NexusDev
Client:	Agile Bridge
University:	University of Pretoria - COS 301 Software Engineering 2026
Document version:	5.2 - Demo 4
Last updated:	28 September 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1 Introduction **4**
1.1 Business Need . . . 4
1.2 Platform Overview . . . 4
1.3 Scope . . . 5
1.4 Constraints . . . 7
1.5 Definitions and Abbreviations . . . 8

2 User Stories / User Characteristics **11**
2.1 Actors . . . 11
2.2 User Stories - Verified Student (Buyer) . . . 11
2.3 User Stories - Verified Student (Seller) . . . 13
2.4 User Stories - Admin . . . 14

3 Use Cases + Use Case Diagrams **16**
3.1 Actor Hierarchy . . . 16
3.2 Use Case Overview . . . 17
3.3 UC0 - Authentication (Base Feature) . . . 18
3.4 UC1 - Create a Textbook Listing . . . 20
3.5 UC2 - View Listing Details . . . 22
3.6 UC3 - Review Listings (Admin) . . . 24
3.7 UC4 - In-App Messaging . . . 25
3.8 UC5 - Edit a Listing . . . 27
3.9 UC6 - Smart Filters and Saved Searches . . . 29
3.10 UC7 - Wishlist . . . 31
3.11 UC8 - Audit Log Query . . . 32
3.12 UC9 - Notification System . . . 34
3.13 UC9.1 - Smart Filter Match Alerts . . . 36
3.14 UC9.2 - Listing Lifecycle Notifications . . . 38
3.15 UC10 - Report, Ban and Appeal Management . . . 40
3.16 UC5.2 - Reserved and Sold Listing Status . . . 42
3.17 UC11 - AI-Assisted Listing Creation . . . 44
3.18 UC12 - Real-Time Textbook Auctions . . . 47
3.19 UC13 - Smart Multi-Book Bundle Optimizer . . . 50

4 Functional Requirements **52**
4.1 AuthService (UC0) . . . 52
4.2 ListingService (UC1) . . . 54
4.3 ModulesService (UC2) . . . 55

4.4 ModerationService (UC3) . . . 56
4.5 MessagingService (UC4) . . . 57
4.6 ListingService Extension (UC5, UC5.2) . . . 57
4.7 SmartFilterService and SavedSearchService (UC6) . . . 58
4.8 WishlistService (UC7) . . . 58
4.9 AuditLogService Extension (UC8) . . . 58
4.10 NotificationService (UC9, UC9.1, UC9.2) . . . 59
4.11 ModerationService Extension (UC10) . . . 60
4.12 VisionService (UC11) . . . 61
4.13 AuctionService (UC12) . . . 62
4.14 BundleService (UC13) . . . 63

5 Non-Functional Requirements 65

6 Domain Model 68
6.1 UML Diagram (Currently, as of Demo 4) . . . 68
6.2 Entity Descriptions . . . 68

7 Appendix 72
7.1 Previous Versions . . . 72

1. Introduction

1.1 Business Need

University students in South Africa struggle to get affordable textbooks every semester. Right now the way people actually buy and sell used textbooks is pretty scattered, most students end up relying on WhatsApp groups, Facebook Marketplace, or just word of mouth. None of these were ever built for trading academic textbooks specifically, so they run into three main problems.

First, it's hard to actually find what you need. Listings don't include structured details like edition, condition, or module code. So if a student is looking for the fifth edition of a specific textbook for a specific module, there's no real way to filter for that. They have to scroll through a lot of irrelevant posts and message multiple sellers, only to find out the edition is wrong after all that.

Second, there's basically no trust built in. There's no way to check that a seller is actually a real university student. Privacy is also an issue, buyers and sellers end up swapping personal contact details with total strangers before they've even agreed on anything, which isn't great for either side.

Third, none of these platforms are built around how university actually works. Nothing understands the academic calendar, module codes, or faculty structure. Students can't browse by module, filter by semester, or find listings that are actually relevant to the course they're taking.

The Uni Textbook Marketplace is our attempt at solving all three of these problems directly, by building something purpose-made, verified, and module-aware for university students.

1.2 Platform Overview

The Uni Textbook Marketplace is a responsive web platform that lets verified university students buy, and/or sell second-hand textbooks in a way that's safer and more structured than what's currently out there, and actually organised around modules. Students register with their university email, which gets verified with a one-time password (OTP). Once they're verified, students can create structured textbook listings with details like ISBN, edition, condition, annotation level, module code, price, and any extra notes. Buyers can then browse listings filtered by faculty, module code, semester, edition, price range, and condition. There's also an admin moderation layer to keep listing quality reasonable, every listing starts out pending and has to be reviewed by an admin before other students can see it. The platform is being built for Agile Bridge and hosted on Microsoft Azure.

1.3 Scope

Delivered for Demo 1 (22 May 2026)

- Student registration with university email domain verification (OTP)
- Student login with JWT-based authentication
- Role-based access control (Student vs Admin)
- Structured textbook listing creation (UC1): ISBN, title, author, edition, condition, annotation level, module code, price, has notes
- Listing detail view for students (UC2): view own listings (approved + pending) and browse others' approved listings
- Admin listing review dashboard (UC3): approve or reject pending listings with soft delete and audit trail
- Module code search-as-you-type lookup, filterable by university
- Basic themes and form validation
- Responsive web interface (mobile + desktop)
- GitHub Actions CI/CD pipeline
- PostgreSQL database with full schema and seed data

Delivered for Demo 2 (31 July 2026)

- Privacy-first in-app messaging (UC4, Firebase Firestore microservice)
- Edit an existing listing while its status is PENDING or REJECTED (UC5)
- Smart search filters (UC6): price range, edition, condition, annotation level, module, faculty
- Saved searches (UC6 extension): a student can save a filter combination and revisit it later
- Wishlist (UC7): a student can save individual listings for later
- Admin audit log (UC8): a queryable, immutable record of every moderation action
- Staging environment with its own database, alongside production, with both auto-deployed from separate branches

Delivered for Demo 3 (31 August 2026)

- Reserved and Sold listing status (UC5.2): an intermediate `RESERVED` state between `APPROVED` and `SOLD`, settable by the seller from the messaging thread or My Listings
- In-app notification system (UC9): notification bell, unread count, and a full notification centre, backed by a single internal `NotificationService.emit()` method
- Smart filter match alerts (UC9.1): a student is notified when a newly created listing matches one of their saved searches
- Listing lifecycle notifications (UC9.2): sellers, admins, and wishlisters are notified on approval, rejection, re-review, and status changes
- Report, ban and appeal management (UC10): students can report a listing, admins can ban the reported seller, and a banned user can submit an appeal for admin review

Delivered for Demo 4 (1–2 October 2026)

Every use case delivered for Demos 1 through 3 (UC0–UC10, UC5.2) is finalised for Demo 4, not still in progress. In addition, three wow factors were built this sprint, in parallel, with no dependencies between them:

- **AI-Assisted Listing Creation (UC11):** an OCR pipeline (Azure AI Vision Read API) that fuzzy-matches a photographed book cover against the existing Book table and pre-fills the listing form, plus a purely client-side background-removal and draggable-corner crop tool. Extends UC1; manual entry always remains available as a fallback.
- **Real-Time Textbook Auctions (UC12):** a seller can convert an approved listing into a time-boxed auction with live bidding, concurrency-safe bid resolution, an anti-sniping extension window, and automatic close with winner determination.
- **Smart Multi-Book Bundle Optimizer (UC13):** given a set of modules, a greedy set-cover-style algorithm recommends the seller grouping that minimises the number of separate meetups, shown alongside the naive cheapest-per-book total for comparison.

Not in scope

- **Payment processing:** the platform only sets up meetup-based exchanges. No payment gateway will be built in any sprint.
- **Mobile application:** a React Native (Expo) app is more of a stretch goal, something we'd only look at once all the core web stuff is actually done.

- **ISBN metadata API lookup:** entries are manual only for now. Hooking up an external ISBN API is not happening yet.
- **Online textbook renting system:** something we flagged as a possible future feature, but it has legal implications we haven't looked into, so it's just stubbed out for now.

1.4 Constraints

Constraint	Detail
Hosting	Provided by Agile Bridge on Microsoft Azure. We don't control the actual infrastructure provider ourselves.
Authentication	OTP email verification is required. Students have to verify their university email before they can access seller and messaging features.
Email allowlist	Limited to student email domains that are confirmed against each university's own student IT documentation. Fifteen South African universities currently qualify. Any university whose student domain is not confirmed is left out until it is. `@tuks.co.za` (University of Pretoria) remains the one actually in active use. See the full table and the reasoning behind it under R1.4. This list can be changed in the backend environment without touching the code.
No payment processing	For now, the whole thing is built around meetup-based exchanges. No payment gateway, escrow, or anything financial is included anywhere.
ISBN metadata	We're not hooking up an external ISBN lookup API for Demo 1. Sellers just type the book details in themselves.
No mobile app in MVP	The platform is a responsive web app only for now. React Native is a stretch goal for later, not part of the MVP.
Firebase credentials	Firebase Firestore credentials are set up and working. The messaging microservice (UC4) is live on both production and staging.

Constraint	Detail
Azure environment access	Both a production and staging environment are live on Azure Container Apps, and both deploy automatically from `main` and `develop` respectively. See the SAS document for the full deployment setup.

1.5 Definitions and Abbreviations

Term	Definition
OTP	One-Time Password. A 6-digit code sent to a student's university email to verify their identity during registration.
JWT	JSON Web Token. A signed, stateless token issued by the backend after successful login. It carries the user's ID and role claim.
UC	Use Case. A numbered interaction between an actor and the system.
UC1	Create a textbook listing (Student).
UC2	View listing details (Student).
UC3	Review listings (Admin).
UC4	Send and receive in-app messages about a listing (Student).
UC5	Edit an existing listing while its status is PENDING or REJECTED (Student).
UC5.2	Extend a listing's status with an intermediate RESERVED state between APPROVED and SOLD (Student).
UC6	Filter listings with additional criteria, and save a filter as a saved search (Student).
UC7	Add or remove a listing from a personal wishlist (Student).
UC8	Query the audit log of moderation actions (Admin).
UC9	Receive, view, and mark as read in-app notifications of relevant platform events (Student).
UC9.1	Alert a student when a newly created listing matches one of their saved searches (Student).

Term	Definition
UC9.2	Notify a seller, admin, or wishlister when a listing's status or review state changes (Student / Admin).
UC10	Report a listing, review and ban a seller, and review a banned user's appeal (Student / Admin).
UC11	Scan a textbook cover photo to auto-fill a listing's fields via OCR, and adjust an auto-detected crop before confirming (Student, Seller).
UC12	Convert an approved listing into a time-boxed auction, bid on it in real time, and have it close automatically with a determined winner (Student).
UC13	Select a set of modules and receive a recommended grouping of sellers that minimises the number of separate meetups needed to get every required textbook (Student, Buyer).
PENDING	The initial status of every new listing. It is not visible to other students until an admin approves it.
APPROVED	A listing that has been reviewed and approved by an admin. It is visible to all verified students.
REJECTED	A listing that has been rejected by an admin. It is soft-deleted and retained in the database with an audit log entry.
RESERVED	An intermediate listing status (UC5.2), set by the seller once a sale has been agreed with a buyer, before the listing is marked SOLD.
SOLD	A terminal listing status (UC5.2), set by the seller once a sale is complete.
SOFT_DELETED	A listing that has been marked as deleted but is retained in the database. The deleted_at column is set.
BANNED	The state of a user account with `is_banned = true` (UC10), set by an admin after reviewing a report. A banned account is rejected with 403 Forbidden on every login or token-refresh attempt until reinstated.
REVERSED	The outcome of an appeal case (UC10) in which an admin reinstates the banned account, setting `is_banned = false`.

Term	Definition
`emit()`	Shorthand for `NotificationService.emit(userId, type, payload)` (UC9), the internal method any backend module calls to create a notification for a user.
OCR	Optical Character Recognition. Used in UC11 to extract text (title, author, ISBN) from a photographed book cover via the Azure AI Vision Read API.
Reserve Price	The minimum accepted winning bid on an auction (UC12). If no bid meets it by the auction's close, the auction ends without a sale.
Anti-sniping	An automatic extension of an auction's `end_time` (UC12) when an accepted bid arrives within a fixed window of the scheduled close, so a last-second bid cannot win purely by timing.
FR	Functional Requirement.
SRS	Software Requirements Specification.
API	Application Programming Interface.
DTO	Data Transfer Object. A class used to define and validate the shape of request and response payloads.
RBAC	Role-Based Access Control. The system grants permissions based on a user's role (Student or Admin).
WCAG AA	Web Content Accessibility Guidelines Level AA. The accessibility standard the UI must meet.
UP	University of Pretoria.
CI/CD	Continuous Integration and Continuous Deployment. Automated pipelines triggered on every pull request.

2. User Stories / User Characteristics

2.1 Actors

There are three types of actors in the system:

Verified Student (Buyer)
A registered, OTP-verified student who mainly uses the platform to find and buy or swap textbooks. They have a university email from an allowlisted domain. They browse approved listings, filter by module code, faculty, edition, and condition, and reach out to sellers through the in-app messaging system.

Verified Student (Seller)
A registered, OTP-verified student who creates structured textbook listings. They fill in all the required listing details, ISBN, edition, condition, annotation level, module code, and price. They keep an eye on the status of their listings and get notified once an admin approves or rejects what they submitted.

Admin
A platform moderator who checks every pending listing before it becomes visible to students. The admin can approve or reject listings, and a rejection triggers a soft delete plus an audit log entry. The admin can also browse all approved listings the same way a student can.

2.2 User Stories - Verified Student (Buyer)

US-B01
As a verified student buyer, I want to browse all approved textbook listings so that I can find a textbook I need without contacting sellers who do not have what I am looking for.

US-B02
As a verified student buyer, I want to filter listings by module code, faculty, edition, and condition so that I can quickly narrow down the results to exactly what I need for my specific course.

US-B03
As a verified student buyer, I want to view the full details of a listing including ISBN, edition, condition, annotation level, price, and whether notes are included so that I can make an informed decision before contacting the seller.

US-B04
As a verified student buyer, I want to see that every seller is a verified university student so that I can trust that the person I am dealing with is a genuine member of the university

community.

US-B05
As a verified student buyer, I want the platform to only show me listings relevant to my university so that I am not exposed to textbooks from institutions using different editions or curricula.

US-B06
As a verified student buyer, I want to see a notification bell with an unread count and a full notification centre so that I know when something relevant to me has happened without having to keep checking the platform manually.

US-B07
As a verified student buyer, I want to be alerted when a newly created listing matches one of my saved searches so that I can act on textbooks I need as soon as they become available, rather than having to keep re-searching myself.

US-B08
As a verified student buyer, I want to report a listing that I believe violates platform standards so that admins can review it and take action if needed.

US-B09
As a verified student buyer, I want to see whether a listing I am interested in is still available, reserved, or already sold, both on the listing itself and inside my conversation with the seller, so that I do not waste time pursuing a textbook that is no longer available.

US-B10
As a verified student buyer, I want to message a seller directly from a listing's detail page so that I can ask questions or arrange a purchase without sharing my personal contact details outside the platform.

US-B11
As a verified student buyer, I want to filter listings by price range, edition, and annotation level, in addition to module and faculty, so that I can narrow results down even further when several sellers have listed the same textbook.

US-B12
As a verified student buyer, I want to save a combination of filters as a search so that I can re-apply it later without re-entering the same criteria every time I browse.

US-B13
As a verified student buyer, I want to save a listing to a personal wishlist so that I can keep track of textbooks I am interested in without committing to messaging the seller straight away.

US-B14
As a verified student buyer, I want to place a bid on an auctioned listing and see the current price and time remaining update in real time so that I can compete fairly for a textbook without having to keep refreshing the page.

US-B15
As a verified student buyer, I want to select all the modules I am taking and get a recommended grouping of sellers that minimises how many separate people I have to meet up with, compared against the plain cheapest-per-book total, so that I can decide whether the convenience of fewer meetups is worth any extra cost.

2.3 User Stories - Verified Student (Seller)

US-S01
As a verified student seller, I want to create a structured textbook listing with fields for ISBN, title, author, edition, condition, annotation level, module code, price, and whether notes are included so that buyers can find exactly what they need without having to ask clarifying questions.

US-S02
As a verified student seller, I want to see my listings that are pending admin review, which are clearly marked with a "Pending Review" badge so that I know my listing has been submitted successfully and understand why it is not yet visible to other students.

US-S03
As a verified student seller, I want to indicate whether my listing includes supplementary study notes so that buyers who need additional study materials can identify and prioritise my listing.

US-S04
As a verified student seller, I want to view all my active approved listings in one place so that I can track what I currently have listed and manage my inventory.

US-S05
As a verified student seller, I want to be notified as soon as an admin approves or rejects my listing so that I know its outcome without repeatedly checking my listings page.

US-S06
As a verified student seller, I want to mark a listing as Reserved once I have agreed a sale with a buyer, and then as Sold once the sale is complete, so that other buyers are not misled into contacting me about a textbook that is no longer available.

US-S07
As a verified student, I want to submit an appeal if my account is banned so that I have

a fair opportunity to explain my side and have the decision reviewed rather than losing access to the platform permanently without recourse.

US-S08
As a verified student seller, I want to reply to buyers who message me about my listings so that I can answer their questions and agree on a sale without exchanging personal contact details.

US-S09
As a verified student seller, I want to edit the details of a listing that is still pending or was rejected so that I can correct a mistake or update its condition and price without creating a brand new listing from scratch.

US-S10
As a verified student seller, I want an approved listing to stay locked from editing so that buyers can trust that what an admin reviewed is what is actually being offered.

US-S11
As a verified student seller, I want to photograph my textbook's cover and have the title, author, and edition fields pre-filled automatically so that I do not have to retype details that are already printed on the book, while still being able to review and correct every field before submitting.

US-S12
As a verified student seller, I want to convert one of my approved listings into a time-boxed auction with a reserve price so that buyers competing for a high-demand textbook can determine its price themselves, rather than me guessing a fixed price upfront.

2.4 User Stories - Admin

US-A01
As an admin, I want to see a dashboard of all listings currently in PENDING status so that I can review new submissions and ensure they meet platform standards before they become visible to students.

US-A02
As an admin, I want to approve a pending listing with a single action so that verified, accurate listings become available to buyers as quickly as possible without unnecessary delays.

US-A03
As an admin, I want to reject a pending listing with an optional note explaining the reason so that the seller understands why their listing was not approved and can correct the issue and resubmit.

US-A04
As an admin, I want rejected listings to be soft-deleted and retained in the database with a `deleted_at` timestamp and audit log entry so that there is a complete and recoverable record of all moderation actions for accountability purposes.

US-A05
As an admin, I want to browse all approved listings the same way a student can so that I can monitor listing quality and identify any listings that should be removed after approval.

US-A06
As an admin, I want to be notified whenever a seller edits a listing that resets it back to PENDING status so that I know new or re-submitted listings are waiting for my review without having to keep checking the dashboard.

US-A07
As an admin, I want to see a dashboard of all pending reports so that I can review reported listings and either dismiss the report or ban the seller if the report is justified.

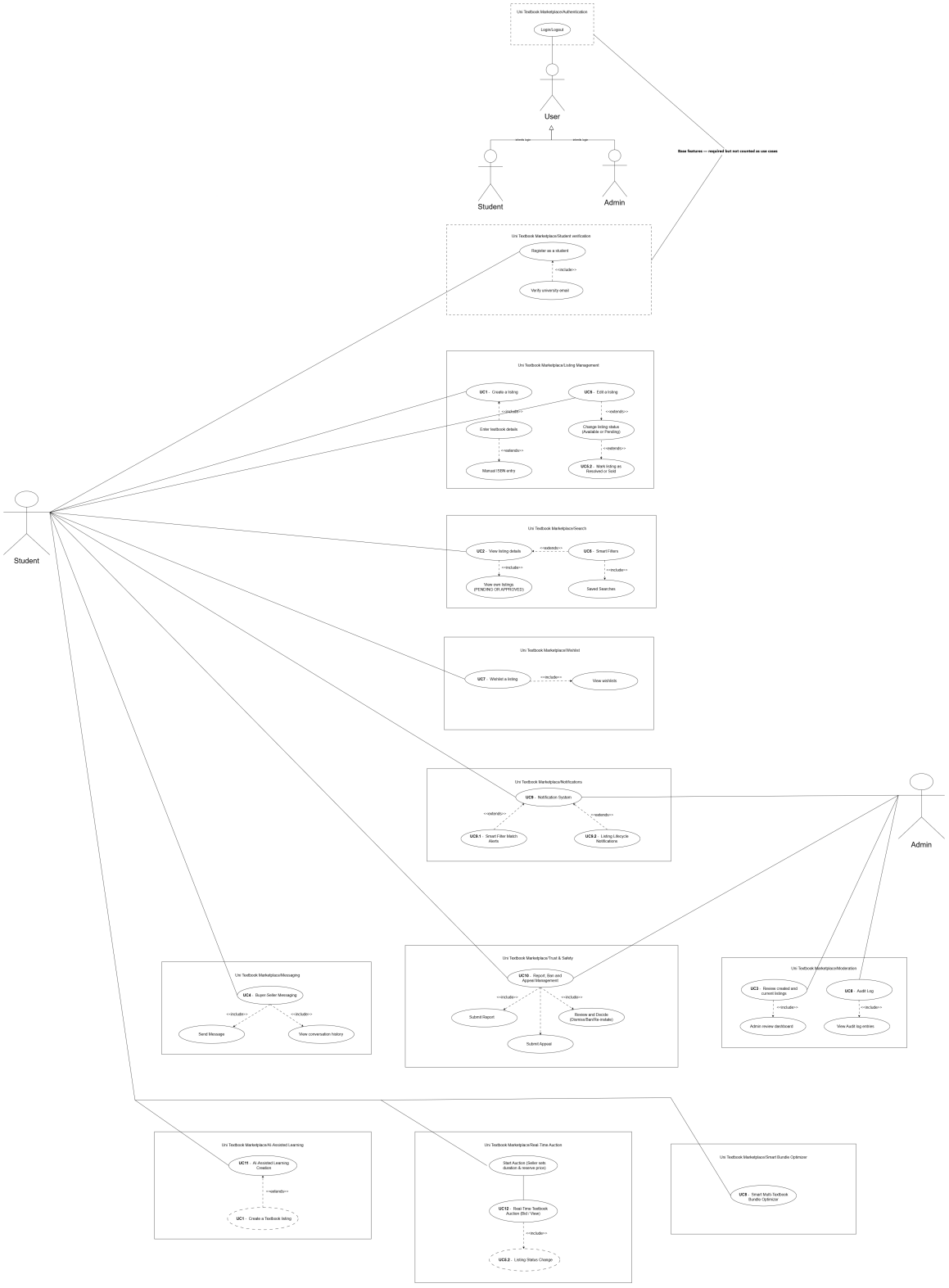
US-A08
As an admin, I want to see a dashboard of all pending appeal cases so that I can review a banned user's explanation and decide whether to uphold the ban or reinstate their account.

US-A09
As an admin, I want to view the full audit log, along with summary statistics and the history for a specific listing, so that I can track past moderation decisions and maintain accountability for every approval, rejection, ban, and appeal outcome.

3. Use Cases + Use Case Diagrams

3.1 Actor Hierarchy

The diagrams below use this actor hierarchy: the base actor is **User**, which specialises into **Verified Student** and **Admin**.



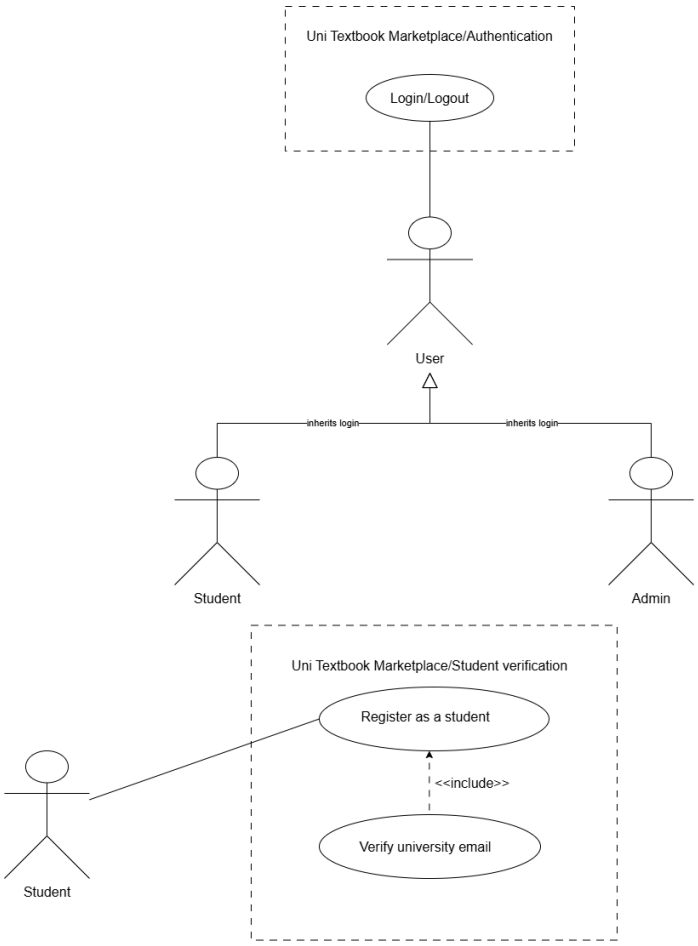
3.2 Use Case Overview

Registration and Login are base features and don't count toward the required number of use cases. UC1 to UC3 were signed off for Demo 1. UC4 to UC8 were delivered for Demo 2. UC5.2, UC9, UC9.1, UC9.2, and UC10 were delivered for Demo 3. UC11, UC12, and UC13 are the three Demo 4 wow factors, built in parallel with no dependencies between them.

ID	Name	Primary Actor
UC0	Authentication (Register + Login + OTP)	Unverified Student (Base feature)
UC1	Create a Textbook Listing	Verified Student (Seller)
UC2	View Listing Details	Verified Student
UC3	Review Listings	Admin
UC4	In-App Messaging	Verified Student
UC5	Edit or Withdraw a Listing	Verified Student (Seller)
UC5.2	Reserved and Sold Listing Status	Verified Student (Seller)
UC6	Smart Filters and Saved Searches	Verified Student (Buyer)
UC7	Wishlist	Verified Student (Buyer)
UC8	Audit Log Query	Admin
UC9	Notification System	Verified Student
UC9.1	Smart Filter Match Alerts	Verified Student (Buyer)
UC9.2	Listing Lifecycle Notifications	Verified Student / Admin
UC10	Report, Ban and Appeal Management	Verified Student / Admin
UC11	AI-Assisted Listing Creation	Verified Student (Seller)
UC12	Real-Time Textbook Auctions	Verified Student (Buyer / Seller)
UC13	Smart Multi-Book Bundle Optimizer	Verified Student (Buyer)

3.3 UC0 - Authentication (Base Feature)

High-Level Description (TUCBW / TUCEW)



This use case begins when an unverified student goes to the registration page and starts the multi-step registration process.

This use case ends when the student has set a password, got a JWT back, and lands on the listings page as an authenticated, verified user.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to Registration page.	System renders Step 1 (personal details: full name, surname).
2	Student enters full name and surname, clicks Next.	System validates non-empty fields and advances to Step 2.

Step	Actor Action	System Response
3	Student enters university name and university email on Step 2.	System validates the email domain against the configured allowlist of confirmed student email domains (see the full table under R1.4).
4	Student submits Step 2.	System generates a 6-digit OTP, stores it in the otps table with a 10-minute expiry, and sends it to the student's university email. System advances to Step 3.
5	Student enters the 6-digit OTP received by email.	System checks the OTP value and expiry again.
6	Student clicks Verify.	System marks the OTP as used, marks the user as verified (`is_verified = true`), and advances to Step 4.
7	Student enters a password and confirms it, then checks the Terms of Service checkbox.	System validates password length (minimum 8 characters) and that both passwords match.
8	Student clicks Register.	System hashes the password, saves the user record to the users table, issues a signed JWT with the student role claim, and redirects to the listings page.

Alternative Flows

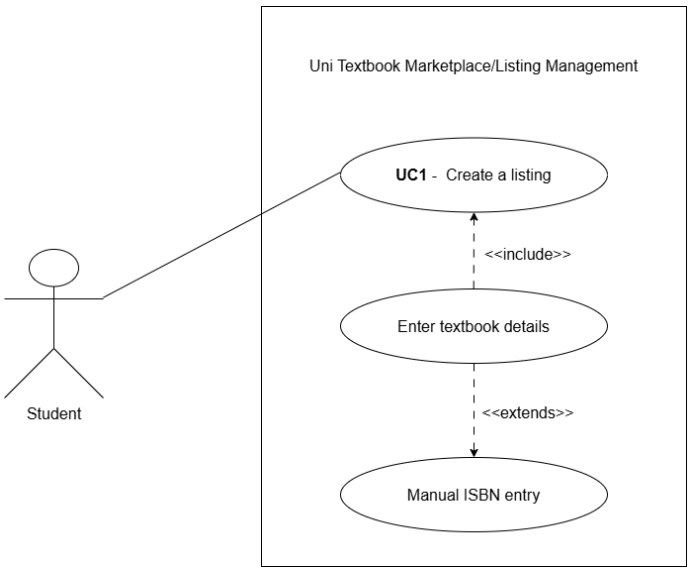
A1 - Invalid email domain: At step 3, if the email domain isn't in the allowlist, the system shows an error saying the email has to end in one of the supported university domains (see R1.4 for the full list) and doesn't move on to Step 3.

A2 - Incorrect OTP: At step 6, if the OTP doesn't match or has expired, the system shows "Invalid or expired OTP" and lets the student ask for a new code.

A3 - Passwords do not match: At step 7, if the two password fields don't match, the system shows "Passwords do not match" and won't submit.

3.4 UC1 - Create a Textbook Listing

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified, logged-in student goes to the Create Listing page and starts filling in the listing form.

This use case ends when the student submits the form and the system creates a new listing with PENDING status, showing a confirmation with a “Pending Review” badge.

Expanded Use Case Table

Step	Actor Action	System Response
1	Authenticated student navigates to Create Listing.	System renders the listing form. The JWT is validated by the JwtAuthGuard.
2	Student enters ISBN.	System accepts the value. ISBN lookup is manual for Demo 1.
3	Student enters title, author, and edition.	System validates that title and edition are non-empty.
4	Student selects condition from dropdown (new, good, fair, poor).	System stores the selected value.
5	Student selects annotation level (none, light, heavy).	System stores the selected value.

Step	Actor Action	System Response
6	Student begins typing a module code in the module search field.	System queries GET /modules?search=[query]&university=UP and displays matching module codes as a dropdown.
7	Student selects a module from the dropdown.	System stores the module_id foreign key.
8	Student enters price in Rands.	System validates that price is a positive decimal value.
9	Student optionally toggles has_notes.	System stores the boolean value.
10	Student optionally uploads one or more photos.	System stores the photo URLs in the photo_url as array.
11	Student clicks Submit.	System sends POST /listings with the JWT in the Authorization header. Backend validates the DTO, creates the listing record with status = PENDING, and returns the new listing ID.
12		System displays a success message: "Your listing has been submitted and is awaiting admin review." The listing is visible on the student's My Listings page with a PENDING badge.

Alternative Flows

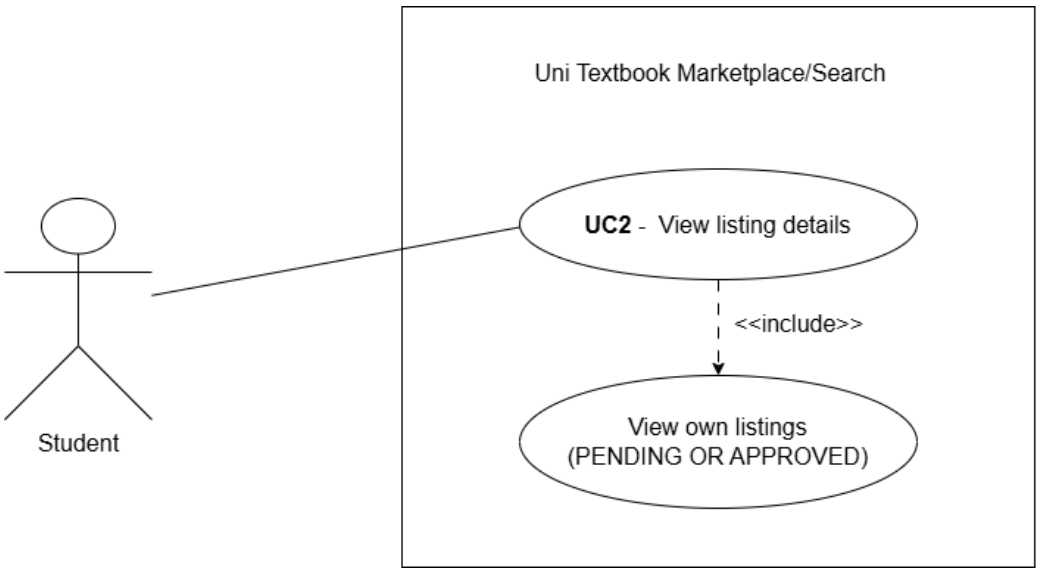
A1 - Not authenticated: At step 1, if there's no JWT or it's expired, the JwtAuthGuard returns 401 and the frontend sends the student to the login page.

A2 - Missing required fields: At step 11, if any required field (title, edition, condition, module, price) is empty, the system returns a 400 Bad Request with field-level validation errors, and the form highlights whichever fields are wrong.

A3 - Invalid price: At step 8, if the price is zero or negative, the system shows "Price must be greater than zero."

3.5 UC2 - View Listing Details

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified, logged-in student goes to the listings page or their own My Listings page.

This use case ends when the student has looked at the full detail page of a specific listing, including all the book metadata, price, condition, annotation level, and listing status.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to Browse Listings.	System calls GET /listings, which returns all APPROVED listings. Each listing card shows title, edition, price, condition, and module code.
2	Student optionally applies filters (module code, faculty, price range, condition).	System re-queries with filter parameters and updates the listing grid.
3	Student clicks a listing card.	System calls GET /listings/:id. The backend returns the full listing object including all fields.

Step	Actor Action	System Response
4		System renders the listing detail page showing: title, author, ISBN, edition, condition, annotation level, has_notes, module code, price, photo carousel, and seller's verified badge.
5	Student navigates to My Listings.	System calls GET /listings/mine. The backend uses the JWT to identify the seller and returns both APPROVED and PENDING listings belonging to that student.
6		System renders the My Listings page. PENDING listings display a "Pending Review" badge. APPROVED listings display an "Approved" badge.

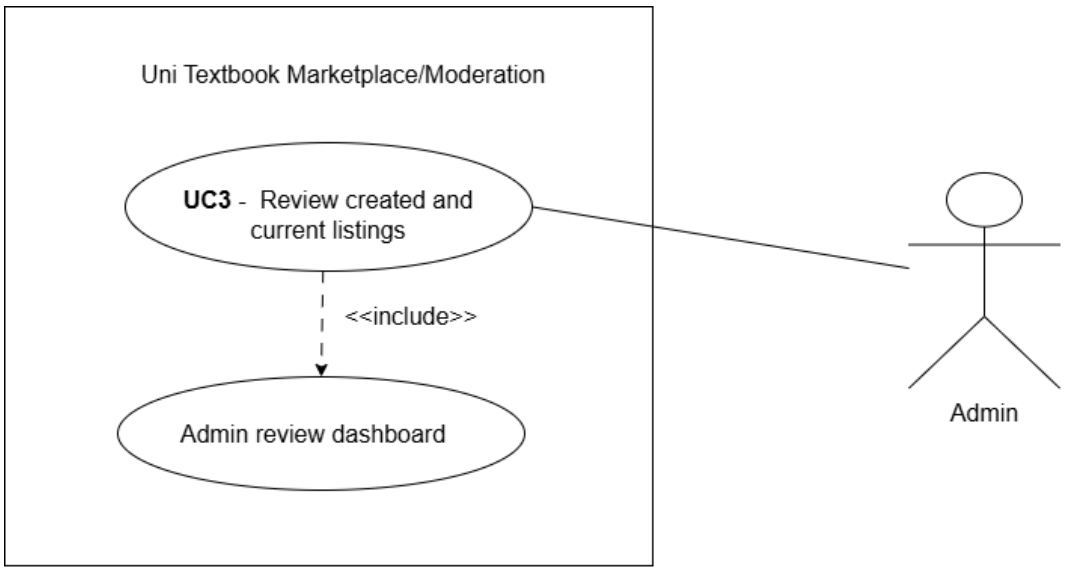
Alternative Flows

A1 - Listing not found: At step 3, if the listing ID doesn't exist, or belongs to a REJECTED or SOFT_DELETED listing, the system returns 404 and shows "Listing not found."

A2 - Unauthenticated access: At step 1, if there's no JWT, the frontend redirects to login. Browsing requires being logged in.

3.6 UC3 - Review Listings (Admin)

High-Level Description (TUCBW / TUCEW)

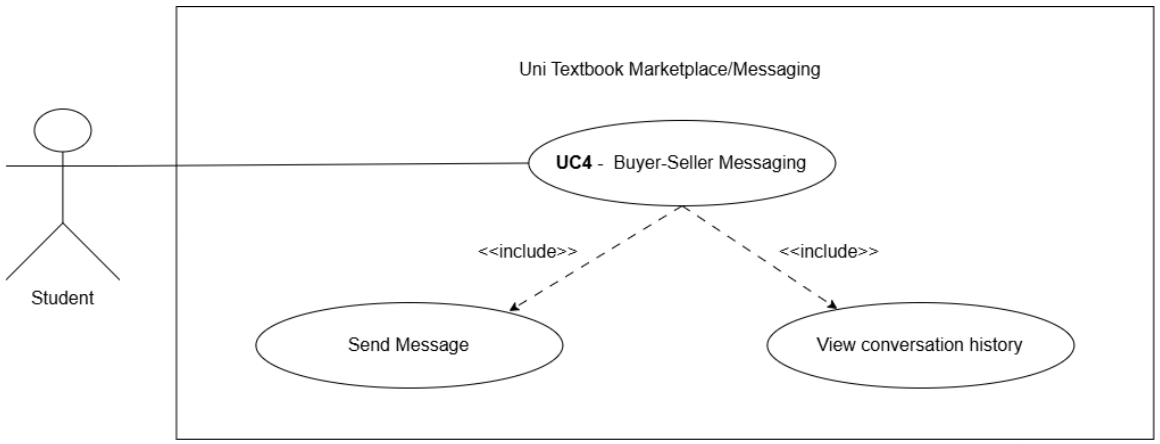


This use case begins when a logged-in admin goes to the Admin Review Dashboard and sees the list of all PENDING listings.

This use case ends when the admin has either approved a listing (status becomes APPROVED and it's now visible to students) or rejected a listing (soft-deleted, with an audit log entry recorded).

3.7 UC4 - In-App Messaging

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, looking at a listing, opens a conversation with the seller.

This use case ends when the two students have exchanged messages about the listing, without either side's personal contact details ever being shared outside the platform.

Messaging runs through a separate Firebase Firestore microservice instead of the core NestJS backend, so that if Firestore ever goes down, it doesn't take listing browsing, creation, or moderation down with it. See the SAS document for the reasoning behind this.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student views a listing detail page and clicks "Message Seller."	System validates the JWT (verified student) and renders the conversation panel.
2		Frontend authenticates directly against Firebase Firestore using the client SDK, scoped to conversations the student is a participant in, independent of the core NestJS backend.
3		If no conversation document already exists between this student and the seller for this listing, Firestore creates a new conversation document.

Step	Actor Action	System Response
4	Student types a message and clicks Send.	Frontend writes the message document directly to the conversation's Firestore collection, storing sender_id, listing_id, message text, and a timestamp.
5		Firestore's real-time listener pushes the new message to the seller's client if they have the conversation open.
6	Seller reads the message and replies.	The reply is written to the same Firestore collection and pushed back to the student in real time.
7	Student or seller reopens the conversation later.	Frontend queries Firestore for all messages tied to the conversation ID, ordered by timestamp, and renders the full history.

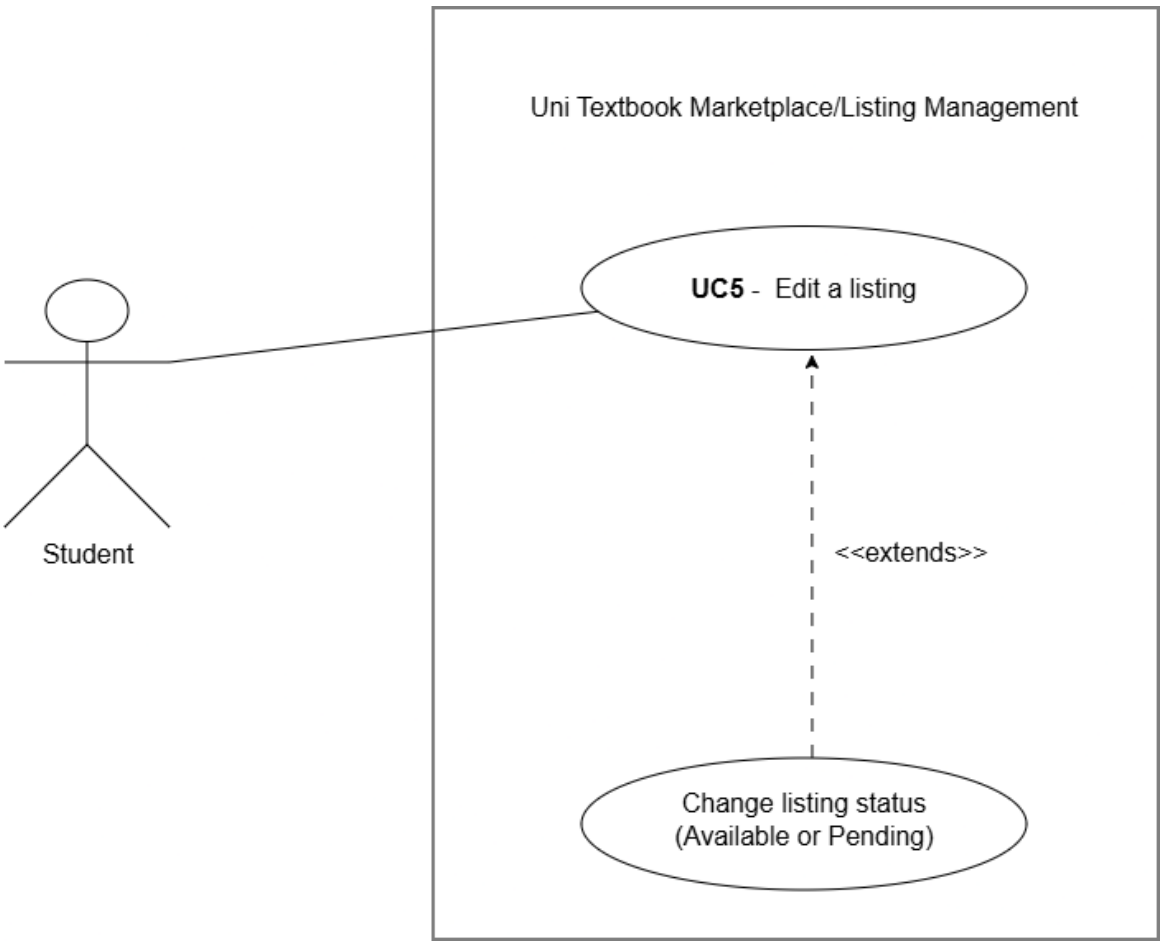
Alternative Flows

A1 - Firestore unavailable: If the client SDK can't connect to Firestore, the system shows "Messaging is temporarily unavailable." Because messaging is kept in its own microservice, listing browsing, creation, and moderation still work fine.

A2 - Not authenticated: If the student's JWT is missing or expired at step 1, the frontend sends them to the login page before the conversation panel even loads.

3.8 UC5 - Edit a Listing

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, looking at one of their own listings on the My Listings page, decides to edit its details.

This use case ends when the listing's details are updated and saved, and the listing's `listing_status` is set back to `PENDING` so an admin reviews it again.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student navigates to My Listings.	System calls GET /listings/mine and renders each listing with Edit and Withdraw actions.
2	Student clicks Edit on a listing.	System checks the listing's status. Editing is only permitted while status is `PENDING` or `REJECTED`; the Edit action is disabled for `APPROVED` listings.

Step	Actor Action	System Response
3		System renders the edit form pre-filled with the listing's current details.
4	Student updates one or more fields (price, condition, annotation level, module, description, photos).	System validates the updated fields using the same rules as listing creation.
5	Student clicks Save.	System sends the update to the backend. If the listing's prior status was `REJECTED`, the system resets it to `PENDING` for re-review. If it was `PENDING`, it remains `PENDING`.
6		System confirms the update and displays the listing's current status badge.

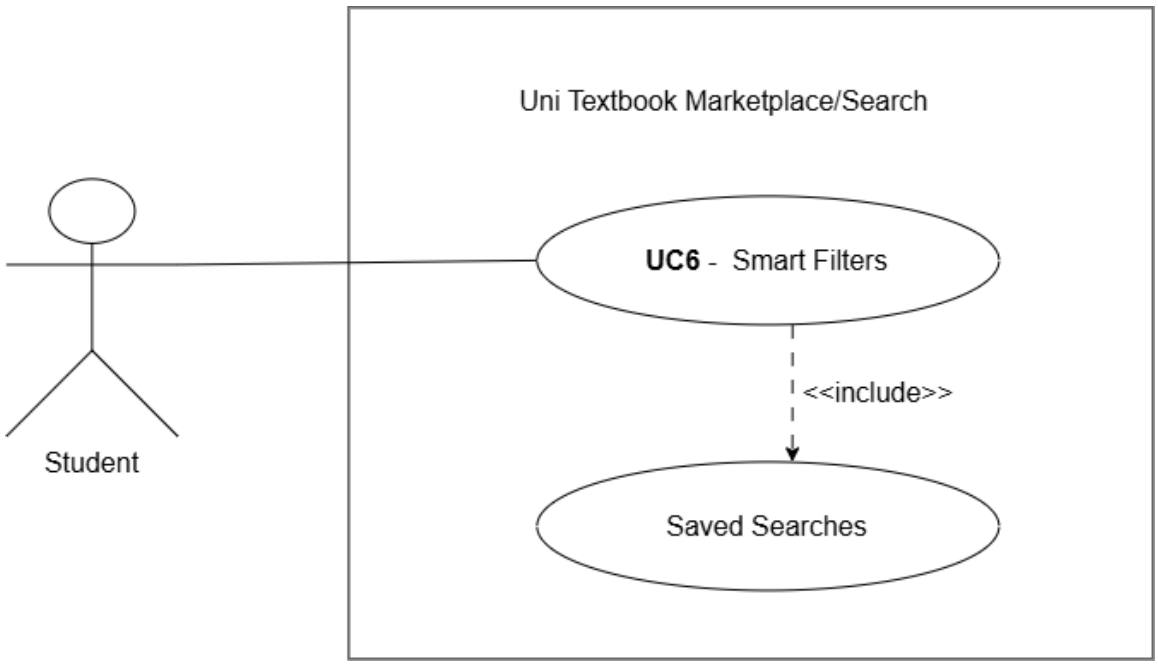
Alternative Flows

A1 - Attempt to edit an APPROVED listing: At step 2, the Edit option isn't available for `APPROVED` listings. If someone tries this directly against the API anyway, the backend returns 403 Forbidden.

A2 - Student cancels edit: At step 4, if the student navigates away without hitting Save, nothing gets saved.

3.9 UC6 - Smart Filters and Saved Searches

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, browsing listings, applies one or more extra filters on top of the Demo 1 module/faculty filter (price range, edition, condition, annotation level).

This use case ends when either the filtered results show up, or if the student chooses to save the current filter combination, a saved search record gets created that they can come back to later without re-typing the same filters.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student on the Browse Listings page opens the extended filter panel.	System renders additional filter controls for price range, edition, condition, and annotation level, alongside the existing module/faculty filters.
2	Student sets a minimum and maximum price.	System stores the selected range as pending filter parameters.
3	Student selects an edition, condition, and/or annotation level.	System stores the selected values as pending filter parameters.

Step	Actor Action	System Response
4	Student clicks Apply Filters.	System calls GET /listings with the combined filter parameters and updates the listing grid with matching results.
5	Student clicks Save Search on the current filter combination.	System prompts for an optional label for the saved search.
6	Student confirms.	System stores the filter combination as a `filter_json` object tied to the student's user ID in the saved_searches table.
7	Student later navigates to Saved Searches.	System calls GET /saved-searches and returns the student's list of saved filter combinations.
8	Student selects a saved search.	System re-applies the stored filter parameters and re-queries GET /listings, updating the grid accordingly.

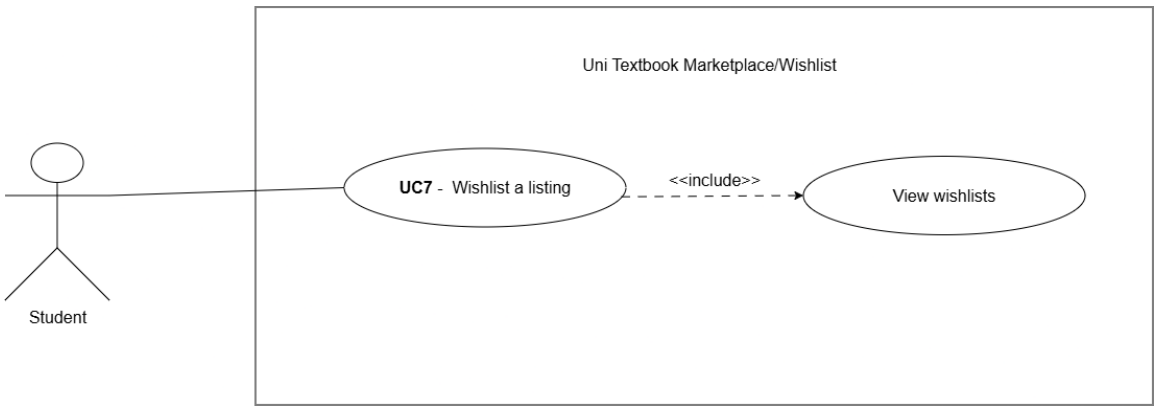
Alternative Flows

A1 - No listings match the filters: At step 4, if nothing matches the combined filters, the system shows "No listings match your filters" and suggests clearing one or more of them.

A2 - Save attempted with no filters applied: At step 5, if no filters have actually been set, the Save Search button is disabled.

3.10 UC7 - Wishlist

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, looking at a listing they're interested in but not ready to message the seller about yet, saves it to their wishlist instead.

This use case ends when the listing shows up on the student's personal wishlist page, where it can also be removed again.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student views a listing detail page and clicks the wishlist (bookmark) icon.	System sends POST /wishlist with the listing ID and the JWT in the Authorization header.
2		Backend creates a wishlist entry using a composite primary key of user_id and listing_id.
3		System updates the icon to a "saved" state on the listing card and detail page.
4	Student navigates to their Wishlist page.	System calls GET /wishlist/mine, which returns the student's wishlist entries joined with their listing details.
5		System renders the wishlist page showing each saved listing's title, price, condition, and status.
6	Student clicks the remove icon on a wishlisted listing.	System sends DELETE /wishlist/:listingId.

Step	Actor Action	System Response
7		Backend removes the wishlist entry and the listing disappears from the page.

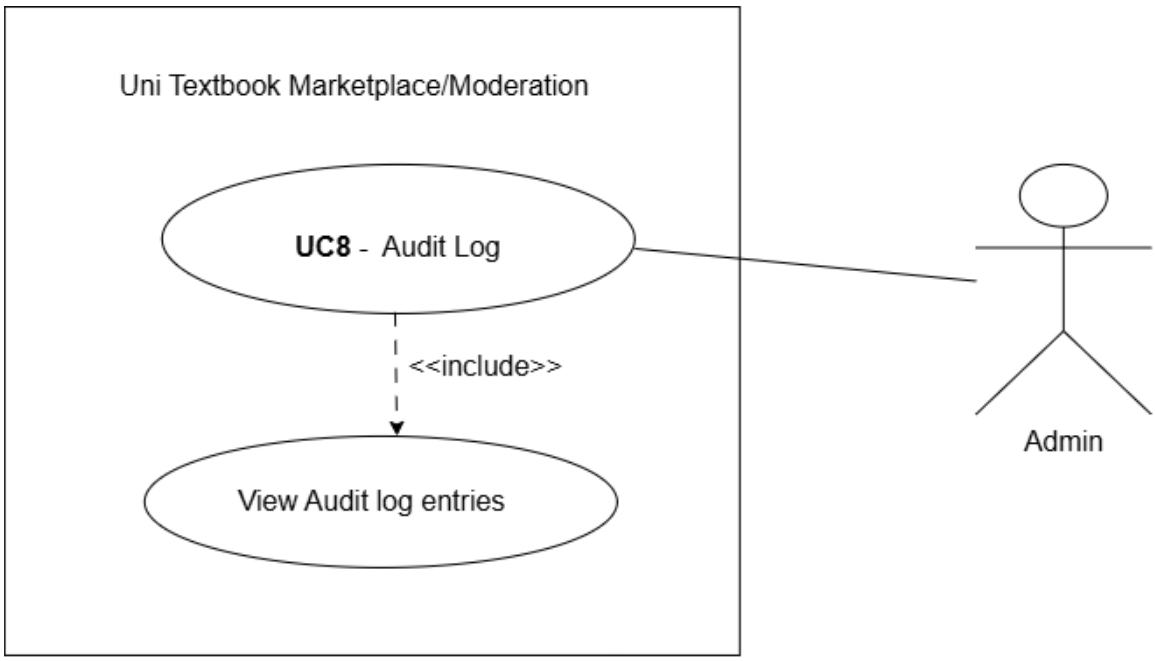
Alternative Flows

A1 - Listing already wishlisted: At step 1, if the listing is already on the student's wishlist, the icon already shows the "saved" state, and clicking it removes the entry instead.

A2 - Wishlisted listing is later deleted: If the listing itself gets deleted, the wishlist entry disappears automatically through `ON DELETE CASCADE` on the listing foreign key, so it no longer shows up on the student's wishlist page.

3.11 UC8 - Audit Log Query

High-Level Description (TUCBW / TUCEW)



This use case begins when a logged-in admin navigates to the audit log view to check past moderation actions.

This use case ends when the admin has pulled up either the full list of audit entries, summary statistics, or the audit history for one specific listing.

Expanded Use Case Table

Step	Actor Action	System Response
1	Admin navigates to the Audit Log page.	System calls GET /audit-log. The JwtAuthGuard validates the JWT and confirms the admin role claim.
2		System renders the full list of audit entries, showing entity_type, entity_id, action (APPROVED or REJECTED), performed_by, performed_at, and notes.
3	Admin switches to the Summary Statistics view.	System calls GET /audit-log/summary and returns aggregate counts of actions taken.
4		System renders the summary statistics (e.g. total approvals vs. rejections) as a table or chart.
5	Admin enters an entity type and entity ID to look up a specific listing's history.	System calls GET /audit-log/entity/:type/:id.
6		System returns and renders the full chronological audit history for that specific entity.

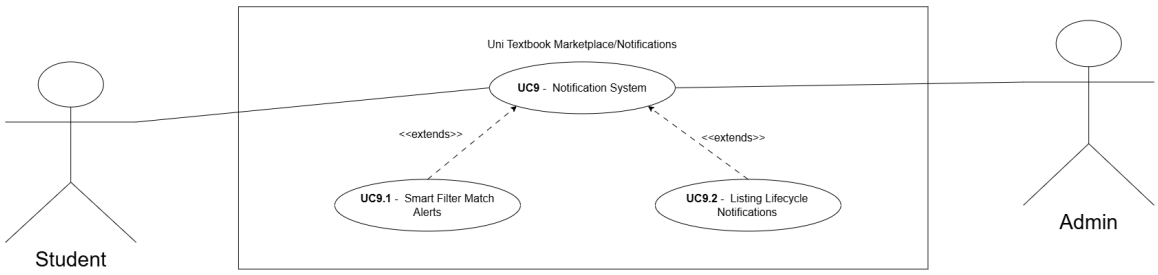
Alternative Flows

A1 - Non-admin access attempt: At step 1, if the logged-in user isn't an admin, the backend returns 403 Forbidden and the frontend doesn't render the audit log view at all.

A2 - No history for entity: At step 5, if there's no audit history for the given entity type and ID, the system shows "No audit history found for this listing."

3.12 UC9 - Notification System

High-Level Description (TUCBW / TUCEW)



This use case begins when any system event relevant to a user occurs (a listing is approved or rejected, a new message arrives, a saved search matches a new listing, a listing needs re-review, a wishlisted listing's status changes, a report is received, a ban is issued, or a case is decided) and the backend calls `NotificationService.emit(userId, type, payload)` for the relevant user.

This use case ends when the notification has been persisted, is visible to the user in both the notification bell dropdown and the full notification centre page, and the user has (optionally) marked it, or all of their notifications, as read.

Expanded Use Case Table

Step	Actor Action	System Response
1	(System-triggered.) A relevant event occurs elsewhere in the platform.	The originating module (e.g. Moderation, Messaging, Smart Filters, Reports) calls `NotificationService.emit(userId, type, payload)`.
2		`emit()` persists a row in the `notifications` table: `user_id`, `type`, `payload` (JSON), `is_read = false`, `created_at`. No `link_url` is stored on the row.
3		For email-worthy notification types, `emit()` additionally sends an email through the existing `mailtrap-email.provider.ts` pathway.
4	Student is logged in with the platform open.	The frontend polls `GET /notifications/mine` every 15 seconds and updates the bell icon's unread-count badge.

Step	Actor Action	System Response
5	Student clicks the bell icon.	System renders the dropdown with the most recent notifications, each showing an icon and copy specific to its `type`.
6	Student clicks a notification.	The frontend resolves the correct destination route from the notification's `type` using a client-side type-to-route lookup map, navigates there, and calls `PATCH /notifications/:id/read`.
7	Student navigates to the full notification centre (`/notifications`) instead.	System calls `GET /notifications/mine` with pagination and renders the full, paginated list.
8	Student clicks Mark All Read.	System calls `PATCH /notifications/read-all`, setting `is_read = true` on every unread notification belonging to that user.

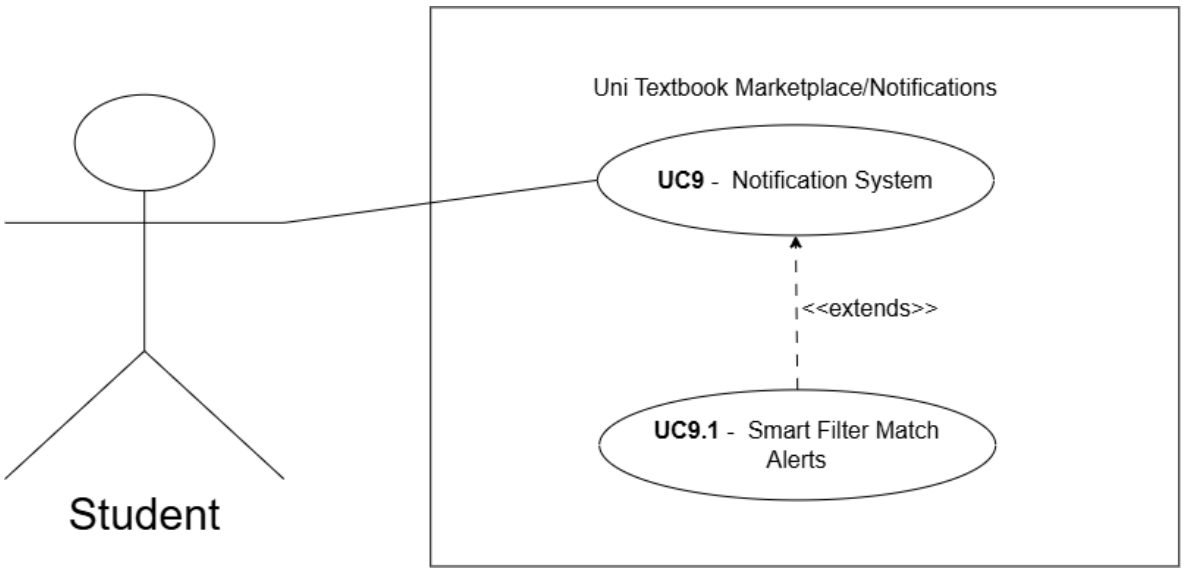
Alternative Flows

A1 - No notifications yet: At step 5 or 7, if the student has no notifications, the system shows an empty state instead of an empty list.

A2 - Notification type missing from the frontend route map: At step 6, if a `type` value has been introduced on the backend but not yet added to the frontend's type-to-route lookup map, the notification still displays and can still be marked read, but clicking it falls back to the notification centre instead of deep-linking to the specific listing, conversation, or case.

3.13 UC9.1 - Smart Filter Match Alerts

High-Level Description (TUCBW / TUCEW)



This use case begins when a student creates a new textbook listing (UC1), extending UC6's Smart Filters and Saved Searches with an alerting step.

This use case ends when every student whose saved search matches the new listing's attributes has received an in-app notification (and an email, where applicable) alerting them to it.

Expanded Use Case Table

Step	Actor Action	System Response
1	A student submits a new listing (UC1, step 11).	In addition to creating the listing, the system queries every saved search whose `filter_json` could plausibly match the new listing's module, faculty, price, edition, condition, and annotation level.
2		Matching treats an unset field on a saved search as "any", so a saved search with only a price range set can still match a listing that didn't specify every other filter field.
3		For every matching saved search, the system calls `NotificationService.emit(userId, 'SMART_FILTER_MATCH', payload)` for the saved search's owner.

Step	Actor Action	System Response
4		`emit()` persists the notification and sends an email, per UC9's flow.
5	Student who saved the search is later logged in, or checks their university email.	The student sees the smart filter match notification in their bell dropdown, notification centre, or inbox.

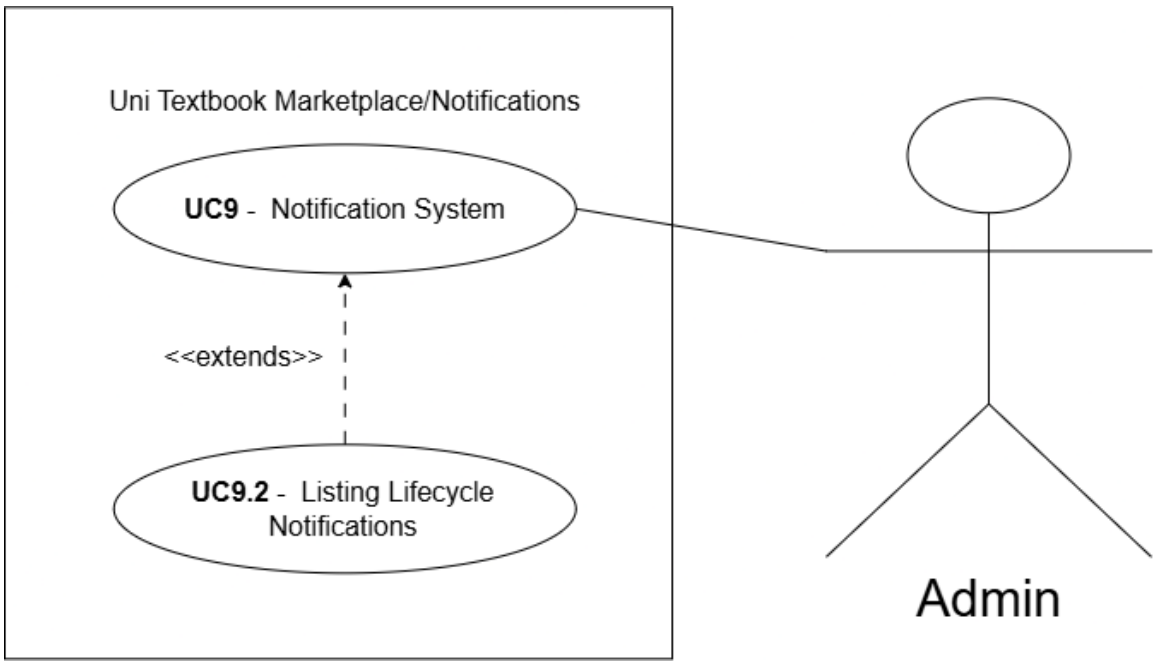
Alternative Flows

A1 - No saved searches match: At step 1, if no saved search matches the new listing's attributes, nothing is emitted and no error occurs.

A2 - Listing created but not yet approved: The match check runs at creation time, before an admin has reviewed the listing. Matched students are alerted to a `PENDING` listing's existence before it is publicly browsable.

3.14 UC9.2 - Listing Lifecycle Notifications

High-Level Description (TUCBW / TUCEW)



This use case begins when one of three listing-lifecycle events occurs: (a) an admin approves or rejects a pending listing, (b) a seller edits a `PENDING` or `REJECTED` listing, resetting it to `PENDING`, or (c) the status of a listing on one or more students' wishlists changes.

This use case ends when the relevant user (the seller, every admin, or every student who has wishlisted that listing) has received an in-app notification for the event, routed through UC9's `emit()`.

Expanded Use Case Table

Step	Actor Action	System Response
1a	Admin approves or rejects a pending listing (UC3, R4.2/R4.3).	In addition to the existing approve/reject logic, the system calls `emit()` for the listing's seller with type `LISTING_APPROVED` or `LISTING_REJECTED`.
1b	Seller edits a `PENDING` or `REJECTED` listing and saves (UC5, step 5).	In addition to resetting the listing to `PENDING`, the system calls `emit()` for every admin, with type `LISTING_NEEDS_REVIEW`.

Step	Actor Action	System Response
1c	A listing's status changes (approved, rejected, reserved, or sold).	The system looks up every wishlist entry referencing that listing and calls `emit()` for each wishlister, with type `WISHLISTED_LISTING_UPDATED`.
2	Seller, admin, or wishlister is logged in.	The relevant notification appears in that user's bell dropdown and notification centre, following UC9's delivery flow.

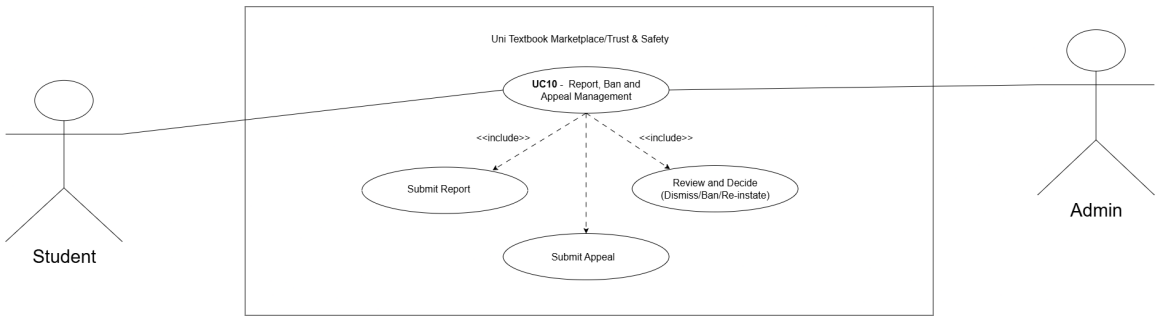
Alternative Flows

A1 - Listing edited multiple times before re-review: At step 1b, each edit that resets a listing to `PENDING` fires a fresh admin notification. There is currently no de-duplication if the same listing is edited several times before an admin reviews it.

A2 - Wishlisted listing deleted rather than status-changed: If the listing itself is deleted, the wishlist entry is removed automatically via `ON DELETE CASCADE` (R8.1.3) before any status change can occur, so no notification is fired in that case.

3.15 UC10 - Report, Ban and Appeal Management

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student, viewing a listing they believe violates platform standards, clicks Report and submits a reason.

This use case ends when either an admin dismisses the report with no further action, or an admin bans the reported listing's seller, that ban is enforced on every subsequent login or token-refresh attempt, and, if the banned user submits an appeal, an admin has reviewed the case and either upheld the ban or reinstated the account.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student views a listing and clicks Report.	System renders a reason form.
2	Student enters a reason and submits.	System sends `POST /reports` with `reporter_id`, `listing_id`, and `reason`. Backend creates a `reports` row with `status = PENDING` and calls `emit()` to confirm receipt to the reporter.
3	Admin navigates to the Report Review dashboard.	System calls `GET /admin/reports` (paginated, filterable by status) and lists all `PENDING` reports.
4	Admin reviews a report and clicks Dismiss.	System calls `PATCH /admin/reports/:id/dismiss`, setting `status = REVIEWED`. No further action is taken.

Step	Actor Action	System Response
5	Admin reviews a report and clicks Ban instead.	System sets `is_banned = true`, `banned_at`, `banned_by`, and `ban_reason` on the reported user, writes an `audit_log` entry with `action = BANNED`, and calls `emit()` to notify the banned user with appeal instructions.
6	Banned user's next login or token-refresh attempt.	A guard checks `is_banned = true` and returns 403 Forbidden with a message pointing to the appeal flow, instead of a generic authentication failure.
7	Banned user submits an appeal.	System sends `POST /cases` with an `appeal_message`. This endpoint is only accessible to users with `is_banned = true`, and creates a `cases` row with `status = PENDING`.
8	Admin navigates to the Case Review dashboard.	System calls `GET /admin/cases` (paginated) and lists all `PENDING` cases.
9	Admin reviews the case and decides to uphold the ban or reinstate the account.	System sets `status = REVERSED` and `is_banned = false` (account restored), writes a corresponding `audit_log` entry, and calls `emit()` to notify the user of the decision.

Alternative Flows

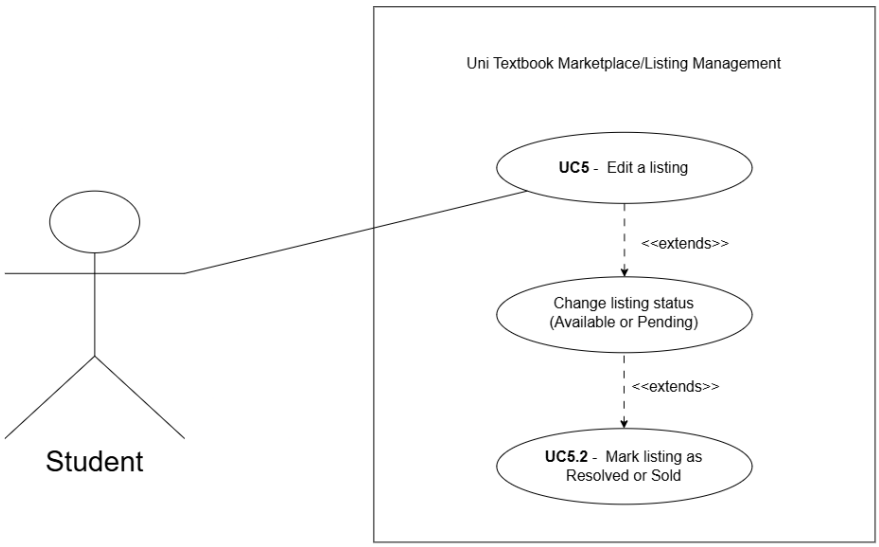
A1 - Non-admin accesses the report or case review endpoints: At steps 3, 4, 5, 8, or 9, the backend returns 403 Forbidden, following the same role-based access control pattern as UC3 and UC8 (R1.5).

A2 - Non-banned user attempts to submit a case: At step 7, if the authenticated user's `is_banned` flag is false, the backend returns 403 Forbidden, since case submission is guarded specifically to banned users.

A3 - Reporting one's own listing: As of this document's writing, the system does not explicitly guard against a student reporting their own listing; this is a known gap rather than an intentional design decision.

3.16 UC5.2 - Reserved and Sold Listing Status

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student seller, having agreed a sale with a buyer through an in-app messaging conversation (UC4) or from the My Listings page, marks the corresponding listing as Reserved. It extends UC5's existing `Available/Pending` status endpoint with a new intermediate `RESERVED` status.

This use case ends when the listing has moved through its extended status lifecycle (`APPROVED` → `RESERVED` → `SOLD`, or directly to `SOLD` as before), with the correct status badge reflected consistently on the listing detail page, My Listings, and the messaging thread.

Expanded Use Case Table

Step	Actor Action	System Response
1	Seller is chatting with a buyer about one of their listings (UC4) and both agree to a sale.	System renders a Reserved/Sold action inside the conversation thread's pinned listing summary card.
2	Seller clicks Mark as Reserved.	System sends `PATCH /listings/:id/status` with `status = RESERVED`. The backend's state-machine transition map validates that the listing's current status permits this transition (`APPROVED` → `RESERVED`) and that the caller owns the listing.

Step	Actor Action	System Response
3		System updates the listing's status to `RESERVED` and returns the updated listing.
4	Seller, from the same thread or from My Listings, later clicks Mark as Sold.	System sends `PATCH /listings/:id/status` with `status = SOLD`. The transition map validates `RESERVED` → `SOLD` (or `APPROVED` → `SOLD` directly, if no buyer was ever reserved).
5		System updates the status to `SOLD`.
6		The listing detail page, My Listings, and the messaging thread all display the current Reserved or Sold badge, sourced from the same underlying status field.

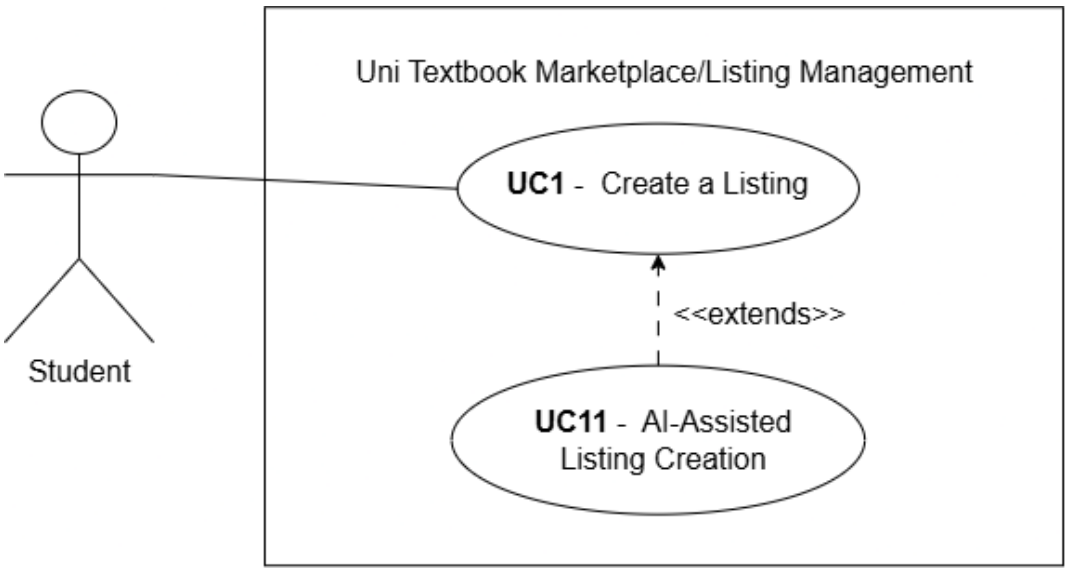
Alternative Flows

A1 - Invalid status transition: At step 2 or 4, if the requested transition isn't permitted by the state-machine map (for example `SOLD` → `RESERVED`), the backend returns 400 Bad Request.

A2 - Non-owner attempts a status change: At step 2 or 4, if the authenticated student doesn't own the listing (and isn't an admin), the backend returns 403 Forbidden.

3.17 UC11 - AI-Assisted Listing Creation

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student seller, on the Create Listing page, chooses "Scan with AI" instead of typing every field manually, and takes or selects a photo of the textbook's cover. It extends UC1 rather than replacing it.

This use case ends when the seller has reviewed the fields pre-filled from the photo (or, if either pipeline is unavailable, fallen back to typing them manually) and adjusted the auto-detected photo crop if needed, then submits the listing exactly as in UC1.

Expanded Use Case Table

Step	Actor Action	System Response
1	Seller clicks "Scan with AI" on the Create Listing page.	System offers "Take photo" (`capture="environment"`) and "Choose from gallery".
2	Seller selects or takes a photo.	System opens the corner-crop editor with the photo and runs `removeUniformBackground` client-side. On a uniform background, the four corner handles are placed on the detected book boundary; otherwise the original photo is shown with handles at default positions and no error.

Step	Actor Action	System Response
3	Seller optionally drags a handle to correct a corner.	System live-updates the crop preview as the handle moves. Each handle is a 44×44px touch target.
4	Seller clicks Confirm Crop.	System draws the image into a canvas clipped to the dragged polygon, fills the area outside it white, and exports a JPEG data URL.
5		System uploads the cropped photo through the existing image-upload flow (Azure Blob Storage), then calls `POST /vision/extract-text` with the uploaded photo's URL.
6		Backend runs Azure AI Vision's Read API on the photo and fuzzy-matches the extracted text against the `Book` table, returning the raw text and, if confident enough, a `matchedBook`.
7		System pre-fills title, author, ISBN, edition, and `bookId` from `matchedBook` on the listing form. Every field remains editable.
8	Seller reviews and corrects the pre-filled fields, then completes the remaining required fields (module, condition, annotation level, price, description) as in UC1.	Same validation and submission as UC1, steps 8–12.

Alternative Flows

A1 - Background not uniform enough to detect corners: At step 2, if `removeUniformBackground` returns no result, the original photo is shown with the four handles at default positions instead of auto-detected ones, with no error message; the seller can still drag every handle manually.

A2 - Extract-text call fails or times out: At steps 5–6, if `POST /vision/extract-text` fails, errors, or does not respond within its timeout, the system shows a brief, non-blocking message ("Auto-fill unavailable, please enter details manually"), keeps the cropped photo,

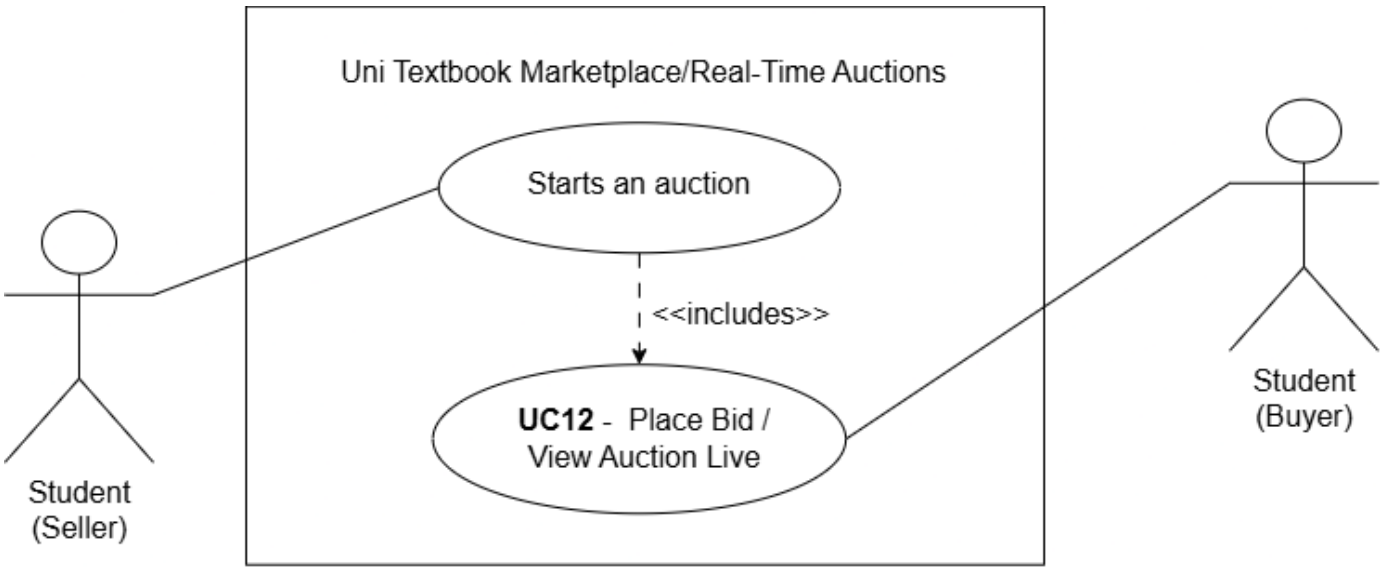
and leaves every field usable exactly as in manual entry (UC1). No part of listing submission is blocked by this failure.

A3 - No confident book match: At step 6, if the OCR text does not fuzzy-match any `Book` record with enough confidence, `matchedBook` is `null` and the fields are left for the seller to enter manually, same as UC1.

A4 - Seller skips the AI path entirely: At step 1, the seller may choose manual entry directly; UC1 proceeds unchanged.

3.18 UC12 - Real-Time Textbook Auctions

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student seller, on one of their own `APPROVED` listings, starts an auction with a duration and an optional reserve price.

This use case ends when the auction reaches its scheduled end time (possibly extended by the anti-sniping rule), the system determines whether the reserve price was met, transitions the underlying listing into the existing Reserved/Sold flow (UC5.2) if there is a winner, and notifies the winner and seller.

Expanded Use Case Table

Step	Actor Action	System Response
1	Seller, on an `APPROVED` listing they own, clicks Start Auction and sets a duration and optional reserve price.	System sends `POST /auctions`, creating an `Auction` row (`status = ACTIVE`) one-to-one with the listing, with `end_time` computed from the chosen duration.
2	Any verified student opens the auction page.	System sends `GET /auctions/:id` for the initial page load (current price, end time, bid history) and opens a Firestore real-time listener for subsequent updates.

Step	Actor Action	System Response
3	A buyer enters a bid at least the minimum increment above the current price and submits.	System sends `POST /auctions/:id/bids`. The backend opens a managed transaction, takes a `SELECT ... FOR UPDATE` row lock on the auction, validates the bid, and records it as `ACCEPTED` or `REJECTED` (with a reason) in the append-only `Bid` table.
4		If accepted and within the extension window of the scheduled close, the system pushes `end_time` forward and increments `extension_count` inside the same transaction, then writes the updated state to Firestore.
5	Every client with the auction page open.	Firestore's real-time listener pushes the updated price, end time, and bid history to every connected client without a page refresh.
6	(System-triggered.) The auction's `end_time` passes with no further accepted bids or extensions.	A scheduled `@Cron()` job detects the expired auction, checks whether the highest accepted bid met the reserve price, and if so transitions the listing through the existing Reserved/Sold flow (UC5.2) and calls `emit()` to notify the winner and seller. If no bid met the reserve, the auction closes with no sale and the listing stays `APPROVED`.

Alternative Flows

A1 - Concurrent bids on the same auction: At step 3, if two bids arrive within milliseconds of each other, the row-level lock serialises them so exactly one is processed at a time; the second bid is validated against the now-current price, never against stale data. This is verified by a dedicated test that fires simultaneous bids and asserts exactly one comes back `ACCEPTED`.

A2 - Seller bids on their own auction: At step 3, the backend rejects the bid with a reason and no state change.

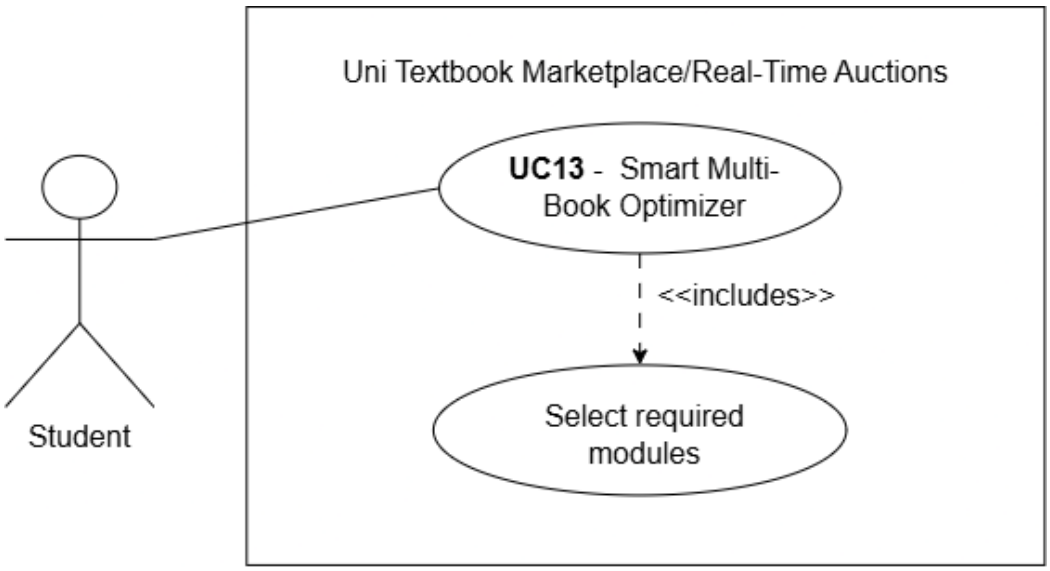
A3 - Bid below the minimum increment: At step 3, the backend rejects the bid with a reason; the current price and end time are unchanged.

A4 - No bids received: At step 6, if the auction closes with zero bids, it ends with no sale.

A5 - Reserve not met: At step 6, if the highest accepted bid is below the reserve price, the auction closes without a sale; the listing stays `APPROVED` and is not transitioned to Reserved/Sold.

3.19 UC13 - Smart Multi-Book Bundle Optimizer

High-Level Description (TUCBW / TUCEW)



This use case begins when a verified student buyer, having selected the modules they are taking, opens the Bundle Optimizer and clicks Find Best Deal.

This use case ends when the student is shown a recommended grouping of sellers covering as many of their required books as possible, alongside the naive cheapest-per-book total, so they can weigh convenience (fewer meetups) against cost.

Expanded Use Case Table

Step	Actor Action	System Response
1	Student opens the Bundle Optimizer and searches for and selects the modules they need textbooks for, reusing the module search-as-you-type component from UC1.	System stores the selected module IDs.
2	Student clicks Find Best Deal.	System sends `POST /bundles/optimize` with the list of module IDs, JWT-protected.
3		Backend finds every required Book across the selected modules via the existing Module↔Book relations, and pulls all currently `APPROVED` listings for those books.

Step	Actor Action	System Response
4		Backend groups candidate listings by seller, scores each seller group by books covered versus price added, and greedily picks the highest-value seller group first, then fills any remaining uncovered books individually from the cheapest listing per book.
5		Backend separately computes the naive cheapest-per-book total (ignoring seller grouping) for comparison.
6		System renders a results page showing the recommended seller groupings, total price, and meetup count, alongside the naive cheapest-per-book total.

Alternative Flows

A1 - No approved listings for a required book: At steps 3–4, a book with zero `APPROVED` listings is shown as uncovered in the results rather than silently dropped.

A2 - No seller overlap between required books: At step 4, if no single seller covers more than one required book, the algorithm falls back cleanly to the naive cheapest-per-book grouping, and both totals shown are equal.

A3 - A single seller covers every required book: At step 4, the algorithm selects that seller's group entirely; the recommended total will typically be lower in meetup count than the naive total, and may be higher or lower in price.

4. Functional Requirements

4.1 AuthService (UC0)

R1: Authentication

R1.1: Student Registration

R1.1.1 The system shall allow a student with a valid university email address to register by providing their full name, surname, university name, and email.

R1.1.2 The system shall reject any email address whose domain is not in the configured allowlist.

R1.2: OTP Generation and Verification

R1.2.1 The system shall generate a cryptographically random 6-digit OTP upon registration and send it to the student's university email address.

R1.2.2 The OTP shall expire after 10 minutes.

R1.2.3 The system shall mark the OTP as used after successful verification to prevent reuse.

R1.3: Login and JWT Issuance

R1.3.1 The system shall authenticate a registered, verified student by validating their email and hashed password.

R1.3.2 On success, the system shall issue a signed JWT containing the user's UUID and role claim (student or admin), to be used on all subsequent authenticated requests.

R1.4: Email Domain Allowlist

R1.4.1 The system shall enforce an email domain allowlist made up only of student email domains that are confirmed against the relevant university's own student IT or student services documentation. Fifteen universities currently qualify (see the table below). `@tuks.co.za` (University of Pretoria) remains the primary domain in active use.

R1.4.2 The allowlist shall be configurable in the backend environment variables without requiring a code change.

R1.4.3 A university whose student email domain has not been confirmed shall be left off the allowlist rather than added on a guess. Eleven South African public universities are currently omitted for this reason: Fort Hare, Sefako Makgatho Health Sciences University, Nelson Mandela University, Venda, Walter Sisulu University, Zululand, the Central University of Technology, Mangosuthu University of Technology, Mpumalanga, Sol Plaatje University, and the Vaal University of Technology. A university is added once its student domain is confirmed. This matters because most universities checked so far run student email on a separate branded domain rather than their general `.ac.za` domain. UP students use `tuks.co.za`, not `up.ac.za`; Wits students use `students.wits.ac.za`, not `wits.ac.za`; and UCT, UJ, UNISA, UWC, CPUT and DUT students all use a different domain from the general one as well. A guess at the general domain is therefore more likely to be wrong than right, and a wrong entry would reject real students at registration. Staff and alumni domains (for example `@uct.ac.za`, `@wits.ac.za` and `@up.ac.za`) are not on the allowlist.

> **Scope note:** this table reflects the SRS's intended allowlist as of this revision. The actual backend `.env` allowlist and the SAS Constraints section have not necessarily been updated to match yet, and a university row also has to exist in the `universities` table before a student from that university can complete registration. Syncing the deployed configuration to this list is tracked as a follow-up rather than assumed already done.

University	**Student Email Domain**	**Verification Status**
Traditional Universities		
University of Cape Town (UCT)	`myuct.ac.za`	Confirmed
University of the Free State (UFS)	`ufs4life.ac.za`	Confirmed
University of KwaZulu-Natal (UKZN)	`stu.ukzn.ac.za`	Confirmed
University of Limpopo (UL)	`myturf.ul.ac.za`	Confirmed (migrated Jul 2025)
North-West University (NWU)	`mynwu.ac.za`	Confirmed
University of Pretoria (UP)	`tuks.co.za`	Confirmed – primary domain in active use
Rhodes University (RU)	`campus.ru.ac.za`	Confirmed

University	Student Email Domain	Verification Status
Stellenbosch University (SU)	`sun.ac.za`	Confirmed – no separate student subdomain found
University of the Western Cape (UWC)	`myuwc.ac.za`	Confirmed
University of the Witwatersrand (Wits)	`students.wits.ac.za`	Confirmed
Comprehensive Universities		
University of Johannesburg (UJ)	`student.uj.ac.za`	Confirmed
University of South Africa (UNISA)	`mylife.unisa.ac.za`	Confirmed – also distance-only, no physical campus or module structure; worth a product-fit discussion, not just a domain check
Universities of Technology		
Cape Peninsula University of Technology (CPUT)	`mycput.ac.za`	Confirmed
Durban University of Technology (DUT)	`dut4life.ac.za`	Confirmed
Tshwane University of Technology (TUT)	`tut4life.ac.za`	Confirmed

R1.5: Role-Based Access Control

R1.5.1 The system shall enforce role-based access control on all protected endpoints using a JWT guard and a role guard.

R1.5.2 Endpoints designated for Admin only shall return 403 Forbidden if accessed with a Student JWT.

4.2 ListingService (UC1)

R2: Listing Management

R2.1: Create Listing

R2.1.1 The system shall allow a verified, authenticated student to create a textbook listing by submitting a DTO containing: ISBN, title, author, edition (integer), condition (new/good/fair/poor), annotation_level (none/light/heavy), module_id (foreign key), price (positive decimal), has_notes (boolean), and photo_urls (array of strings).

R2.1.2 The listing shall be created with status = PENDING.

R2.2: Module Code Search

R2.2.1 The system shall provide a search-as-you-type module code endpoint accepting a search string and an optional university filter.

R2.2.2 The response shall include module code, module name, and faculty.

R2.3: Browse Approved Listings

R2.3.1 The system shall return all listings with status = APPROVED when a verified student calls the browse endpoint.

R2.3.2 Soft-deleted, rejected, and pending listings shall not be returned.

R2.4: View My Listings

R2.4.1 The system shall return all listings belonging to the authenticated student, including both APPROVED and PENDING listings.

R2.4.2 The response shall include the listing status so the frontend can render appropriate badges.

R2.5: View Listing Detail

R2.5.1 The system shall return the full listing detail object for a given listing ID.

R2.5.2 If the listing is REJECTED or SOFT_DELETED, the system shall return 404 Not Found. If the listing is PENDING, it shall only be accessible to its owner or an admin.

4.3 ModulesService (UC2)

R3: Module Management

R3.1: Module Search Endpoint

R3.1.1 The system shall expose a GET /modules endpoint that accepts a search query parameter and an optional university parameter.

R3.1.2 The endpoint shall perform a case-insensitive prefix search on module code and module name.

R3.2: Module Seed Data

R3.2.1 The system shall seed the database with a representative set of University of Pretoria module codes across all faculties. This is fully working now through an idempotent seed script, which we've confirmed runs correctly against both a fresh and an already-populated database.

R3.2.2 The seed data shall include module code, module name, faculty, and semester.

4.4 ModerationService (UC3)

R4: Listing Moderation

R4.1: View Pending Listings

R4.1.1 The system shall return all listings with status = PENDING when an authenticated admin calls the pending listings endpoint.

R4.1.2 The response shall include full listing detail plus the seller's display name.

R4.2: Approve Listing

R4.2.1 The system shall allow an authenticated admin to approve a pending listing. On approval, the system shall set status = APPROVED, reviewed_by = admin UUID, and reviewed_at = current timestamp.

R4.2.2 An audit log entry shall be written with action = APPROVED.

R4.3: Reject Listing with Soft Delete

R4.3.1 The system shall allow an authenticated admin to reject a pending listing with an optional rejection reason.

R4.3.2 On rejection, the system shall set status = REJECTED, deleted_at = current timestamp, reviewed_by = admin UUID, and reviewed_at = current timestamp, and create an audit log entry with action = REJECTED and the optional notes.

R4.3.3 The listing record shall be retained in the database (soft delete, not hard delete).

4.5 MessagingService (UC4)

R5: In-App Messaging

R5.1: Send and Receive Messages

R5.1.1 The system shall allow a verified student to open a conversation with the seller of a listing.

R5.1.2 Messages shall be delivered and stored through the Firebase Firestore microservice, independent of the core backend.

R5.1.3 The frontend shall connect to Firestore directly through the client SDK, rather than through the NestJS backend.

4.6 ListingService Extension (UC5, UC5.2)

R6: Edit, Withdraw, Reserve and Sell Listing

R6.1: Edit Listing

R6.1.1 The system shall allow the owning student to edit the details of their own listing while their listing status is in `PENDING` or `REJECTED`

R6.1.2 Editing an `REJECTED` listing resets its status to `PENDING`.

R6.2: Reserved and Sold Status (UC5.2)

R6.2.1 The system shall expose a status-update endpoint, `PATCH /listings/:id/status`, that allows the owning student to transition their listing to `RESERVED` or `SOLD`.

R6.2.2 The system shall validate every requested status transition against a state-machine transition map before applying it. Permitted transitions are `APPROVED` → `RESERVED`, `RESERVED` → `SOLD`, and `APPROVED` → `SOLD` directly. Any transition not in the map shall be rejected with 400 Bad Request.

R6.2.3 The system shall reject a status-update request with 403 Forbidden if the authenticated student does not own the listing and is not an admin.

R6.2.4 The current status shall be reflected consistently wherever a listing's status is displayed, including the listing detail page, My Listings, and the messaging thread's pinned listing summary card.

4.7 SmartFilterService and SavedSearchService (UC6)

R7: Smart Filters and Saved Searches

R7.1: Extended Filtering

R7.1.1 The system shall allow a student to filter listings by price range, edition, condition, and annotation level, in addition to the module and faculty filters delivered for Demo 1.

R7.2: Save a Search

R7.2.1 The system shall allow a student to save their current filter combination as a saved search, stored as a JSON filter object tied to the student's user ID.

R7.2.2 The system shall allow a student to retrieve their saved searches.

4.8 WishlistService (UC7)

R8: Wishlist

R8.1: Add and Remove Wishlist Entries

R8.1.1 The system shall allow a student to add a listing to their personal wishlist.

R8.1.2 The system shall allow a student to remove a listing from their wishlist.

R8.1.3 A listing shall be removed from a student's wishlist automatically if the listing itself is deleted, since the wishlist entry references the listing by foreign key with `ON DELETE CASCADE`.

R8.2: View Wishlist

R8.2.1 The system shall allow a student to retrieve their full wishlist.

4.9 AuditLogService Extension (UC8)

R9: Audit Log Query

R9.1: Query Audit Log

R9.1.1 The system shall allow an authenticated admin to retrieve the full audit log.

R9.1.2 The system shall allow an authenticated admin to retrieve summary statistics over the audit log.

R9.1.3 The system shall allow an authenticated admin to retrieve the audit history for a specific entity, given its entity type and entity ID.

4.10 NotificationService (UC9, UC9.1, UC9.2)

R10: Notifications

R10.1: Emit and Persist Notifications

R10.1.1 The system shall expose a single internal `NotificationService.emit(userId, type, payload)` method that any backend module can call to notify a user of a relevant event.

R10.1.2 `emit()` shall persist a row in the `notifications` table containing `user_id`, `type`, `payload` (JSON), `is_read = false`, and `created_at`.

R10.1.3 For notification types designated as email-worthy, `emit()` shall additionally send an email through the existing email provider.

R10.2: Retrieve and Read Notifications

R10.2.1 The system shall expose `GET /notifications/mine` to return the authenticated user's notifications, paginated, most recent first.

R10.2.2 The system shall expose `PATCH /notifications/:id/read` to mark a single notification as read, and `PATCH /notifications/read-all` to mark every unread notification belonging to the authenticated user as read.

R10.2.3 The frontend shall poll `GET /notifications/mine` at a fixed interval to keep the notification bell's unread-count badge current while the user has the platform open.

R10.3: Smart Filter Match Alerts (UC9.1)

R10.3.1 When a new listing is created, the system shall check it against every saved search's filter criteria (module, faculty, price, edition, condition, and annotation level), treating any unset filter field as matching any value.

R10.3.2 For every saved search that matches, the system shall call `emit()` for that saved search's owner with type `SMART_FILTER_MATCH`.

R10.3.3 This match check shall run at listing-creation time, independent of the listing's moderation status.

R10.4: Listing Lifecycle Notifications (UC9.2)

R10.4.1 When an admin approves or rejects a pending listing, the system shall call `emit()` for the listing's seller with type `LISTING_APPROVED` or `LISTING_REJECTED` respectively.

R10.4.2 When a seller's edit resets a listing to `PENDING`, the system shall call `emit()` for every admin with type `LISTING_NEEDS_REVIEW`.

R10.4.3 When a listing's status changes, the system shall call `emit()` for every student who has that listing on their wishlist, with type `WISHLISTED_LISTING_UPDATED`.

4.11 ModerationService Extension (UC10)

R11: Reports, Bans and Appeals

R11.1: Submit a Report

R11.1.1 The system shall allow a verified, authenticated student to submit a report against a listing by sending `POST /reports` with a `listing_id` and a `reason`.

R11.1.2 The system shall create a `reports` row with `status = PENDING` and call `emit()` to confirm receipt to the reporter.

R11.2: Review Reports

R11.2.1 The system shall expose `GET /admin/reports`, paginated and filterable by status, restricted to admins.

R11.2.2 The system shall allow an admin to dismiss a report via `PATCH /admin/reports/:id/dismiss`, setting `status = REVIEWED` with no further action taken.

R11.2.3 The system shall allow an admin to ban the reported listing's seller. On ban, the system shall set `is_banned = true`, `banned_at`, `banned_by`, and `ban_reason` on the user, write an audit log entry with `action = BANNED`, and call `emit()` to notify the banned user with appeal instructions.

R11.3: Enforce a Ban

R11.3.1 The system shall check `is_banned = true` on every login and token-refresh attempt and return 403 Forbidden with a message pointing to the appeal flow if the account is banned.

R11.4: Submit and Review an Appeal

R11.4.1 The system shall allow a banned user, and only a banned user, to submit an appeal via `POST /cases` with an `appeal_message`, creating a `cases` row with `status = PENDING`.

R11.4.2 The system shall expose `GET /admin/cases`, paginated, restricted to admins.

R11.4.3 The system shall allow an admin to decide a case, setting `status = UPHELD` (ban remains) or `status = REVERSED` together with `is_banned = false` (account restored), writing a corresponding audit log entry, and calling `emit()` to notify the user of the decision.

R11.5: Role-Based Access Control

R11.5.1 All report-review and case-review endpoints shall be restricted to admins and return 403 Forbidden otherwise, consistent with R1.5.

4.12 VisionService (UC11)

R12: AI-Assisted Listing Creation

R12.1: OCR Auto-Fill

R12.1.1 The system shall expose `POST /vision/extract-text`, restricted to authenticated students, accepting the URL of a photo already uploaded via the existing image-upload flow.

R12.1.2 The system shall call Azure AI Vision's Read API to extract text from the photo, and shall fuzzy-match the extracted text against the `Book` table rather than requiring an exact match.

R12.1.3 The Azure Vision endpoint and key shall be stored only as backend secrets and shall never be exposed to the client, consistent with the existing Blob Storage credential pattern.

R12.1.4 A confident match shall pre-fill the listing form's title, author, ISBN, edition, and `bookId` fields; every pre-filled field shall remain editable by the seller.

R12.2: Background Removal and Adjustable Crop

R12.2.1 The system shall attempt to remove a uniform background from the photo and detect the book's four corner points client-side, with no backend call required for

this step.

R12.2.2 The system shall render the four corners as independently draggable handles over the photo, and shall let the seller adjust any corner before confirming the crop.

R12.2.3 If background removal does not return a confident result, the system shall show the original photo with the handles at default positions rather than blocking the seller or showing an error.

R12.3: Graceful Fallback

R12.3.1 If `POST /vision/extract-text` fails, errors, or does not respond within its client-side timeout, the system shall show a non-blocking message and leave every listing field editable for manual entry, with zero blocking of listing submission.

4.13 AuctionService (UC12)

R13: Real-Time Textbook Auctions

R13.1: Create and View an Auction

R13.1.1 The system shall allow the owning student to convert one of their own `APPROVED` listings into an auction via `POST /auctions`, with a chosen duration and an optional reserve price.

R13.1.2 The system shall expose `GET /auctions/:id` returning the auction's full current state (current price, end time, bid history) for the initial page load.

R13.2: Place a Bid

R13.2.1 The system shall expose `POST /auctions/:id/bids`, processed inside a manually managed database transaction that takes a pessimistic `SELECT ... FOR UPDATE` row lock on the auction before validating or accepting the bid.

R13.2.2 The system shall reject a bid that does not exceed the current price by at least the minimum increment, that is placed by the auction's own seller, or that is placed after the auction has closed, recording the bid and its rejection reason in the append-only `Bid` table.

R13.2.3 For any two bids submitted concurrently against the same auction, exactly one shall resolve as `ACCEPTED`; every bid, accepted or rejected, shall be retained as an immutable row and never updated or deleted.

R13.3: Anti-Sniping Extension

R13.3.1 If an accepted bid arrives within a fixed window of the auction's scheduled `end_time`, the system shall push `end_time` forward and increment `extension_count`, within the same transaction as the bid that triggered it.

R13.4: Auto-Close and Winner Determination

R13.4.1 A scheduled job shall detect auctions whose `end_time` has passed with no further extension, and shall determine a winner only if the highest accepted bid meets the reserve price.

R13.4.2 On a determined winner, the system shall transition the underlying listing through the existing Reserved/Sold status flow (R6.2) and call `emit()` to notify the winner and seller. If the reserve was not met, the listing shall remain `APPROVED` and no transition shall occur.

R13.5: Real-Time Broadcast

R13.5.1 On every accepted bid and every anti-sniping extension, the system shall write the auction's updated state to Firestore, reusing the same Postgres-source-of-truth, Firestore-broadcast pattern established for in-app messaging (UC4), so every connected client's view updates without a page refresh.

4.14 BundleService (UC13)

R14: Smart Multi-Book Bundle Optimizer

R14.1: Compute an Optimized Bundle

R14.1.1 The system shall expose `POST /bundles/optimize`, restricted to authenticated students, accepting a list of module IDs.

R14.1.2 The system shall find every required `Book` across the given modules, pull all currently `APPROVED` listings for those books, and group candidate listings by seller.

R14.1.3 The system shall greedily select the highest-value seller group first (by books covered versus price added), then fill any remaining uncovered books individually from the cheapest listing per book.

R14.2: Comparison Against Naive Cheapest-Per-Book

R14.2.1 The system shall separately compute the naive cheapest-per-book total, ignoring seller grouping, and return both totals so the trade-off between convenience and cost is shown explicitly rather than hidden.

5. Non-Functional Requirements

Each requirement below is written so that a specific, existing test, metric, or measured value in the project already tells us whether it's being met, rather than as an unquantified aspiration.

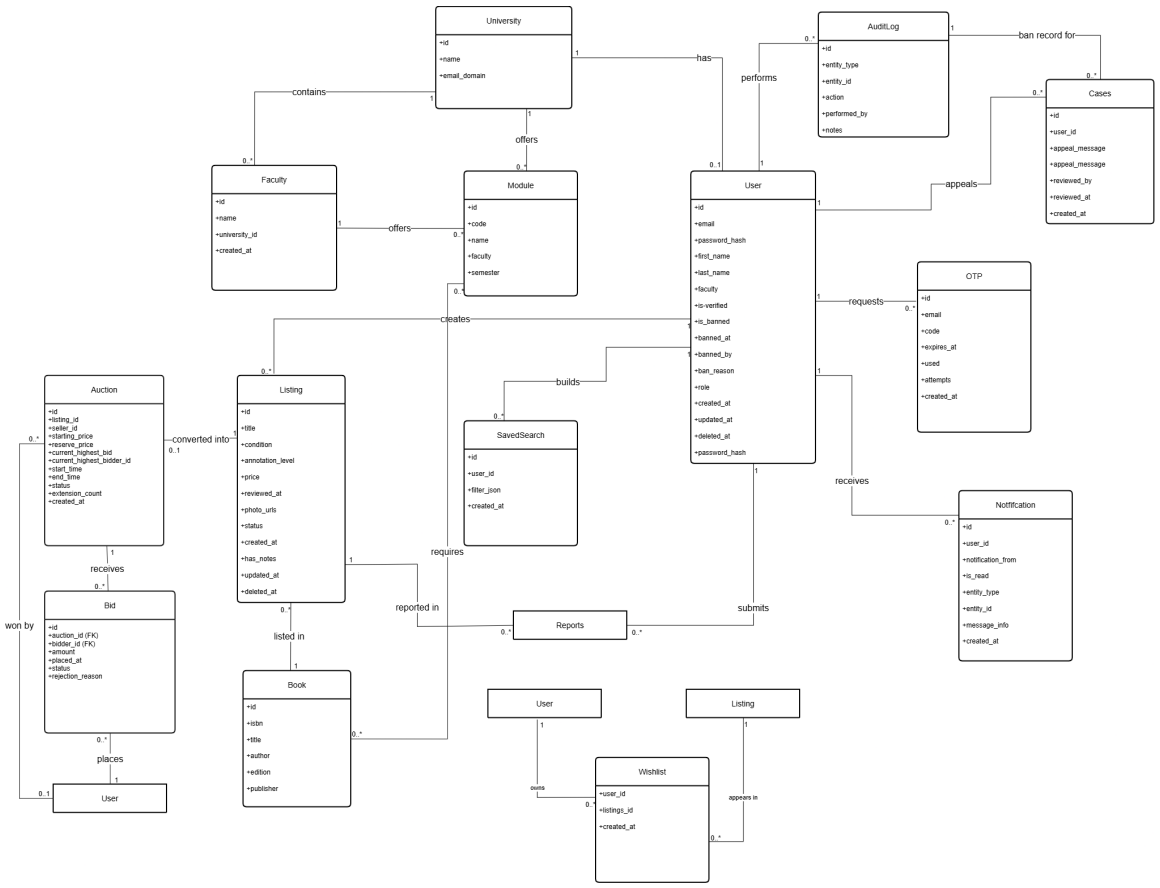
Quality Attribute	Requirement and Quantification
Performance	`GET /listings` search and browse requests shall return within 2 seconds for at least 95% of requests under normal load, defined as up to 50 concurrent users (the expected peak for a single-cohort capstone deployment). New-notification delivery via the bell icon shall reflect a triggering event within one polling interval, currently fixed at 15 seconds. `POST /vision/extract-text` (UC11) shall complete within 5 seconds for at least 90% of requests, backed by a client-side timeout that triggers the manual-entry fallback beyond that. `POST /bundles/optimize` (UC13) shall complete within 2 seconds for a typical module selection (up to 6 modules). Evidence for both new targets is being collected as part of this sprint's NFR testing pass and is not yet captured here.
Scalability	The stateless, JWT-based API layer shall support horizontal scaling without code changes, since no session state is held on the server. Production runs with `min-replicas: 1`, `max-replicas: 2` on Azure Container Apps. Staging scales to zero when idle (`min-replicas: 0`), accepting a cold-start delay of approximately 20 seconds on the first request after an idle period, as a documented cost/availability trade-off rather than a defect.

Quality Attribute	Requirement and Quantification
Security	All protected endpoints shall enforce JWT-based authentication and role-based access control; an admin-only endpoint shall return 403 Forbidden for a Student-role JWT. Passwords are hashed with bcrypt at a cost factor of 12. Login, registration, and password-change endpoints are rate-limited to 5 requests per minute per client. Access tokens expire after 15 minutes, refresh tokens after 7 days. A banned user (`is_banned = true`) shall be rejected with 403 Forbidden on every login or token-refresh attempt from the moment the ban is issued, with no grace period.
Reliability / Availability	Core features (authentication, listings, moderation, reporting, and bans) shall remain fully available if the external messaging microservice (Firestore) is unavailable. Production maintains at least 1 warm replica at all times to avoid cold-start latency for real users. A failed deployment shall be rollback-able to the last known-good revision via Azure Container Apps' revision traffic shifting, without a rebuild, in under 5 minutes; this has been exercised in a real production incident, not only tested in principle.
Maintainability	Backend test coverage shall be measured on every push and pull request via an automated CI pipeline running against a real, ephemeral PostgreSQL container (not a mock), with results tracked through Codecov. 80% automated coverage is the working target; per the Testing Policy, this is not yet an enforced hard gate, merges are currently gated on the test suite passing and SonarCloud's Quality Gate. Every new endpoint shall have a corresponding entry added to the shared `openapi.yaml` the same day it is committed.

Quality Attribute	**Requirement and Quantification**
Accessibility	The UI shall meet a Google Lighthouse Accessibility score of at least 85/100 on every audited page, and at least 90/100 on any page introduced from Demo 3 onward. Most recent measured scores: Landing Page 94, Create Listing 94, In-App Messaging 95, Wishlist 96, Listings Browse 89, and Audit Log 85 (the current lowest scorer, tracked as a follow-up item due to its dense tabular content).
Concurrency (new for Demo 4)	For any two bids submitted concurrently against the same auction (UC12, R13.2), the system shall resolve exactly one as `ACCEPTED`, enforced by a database-level pessimistic row lock (`SELECT ... FOR UPDATE`) rather than application-level checking alone. This is verified by a dedicated integration test that fires simultaneous bids via `Promise.all` against a real test database and asserts a single, deterministic winner.
Auditability	Every moderation, ban, and appeal-decision action (approve, reject, ban, uphold, reinstate) shall write exactly one immutable `audit_log` entry recording the action, the performing admin's UUID, and a timestamp. This is verified by an integration test for each of UC3, UC8, and UC10 that asserts a matching `audit_log` row exists after the action completes.

6. Domain Model

6.1 UML Diagram (Currently, as of Demo 4)



6.2 Entity Descriptions

University

Represents a university registered on the platform. Stores the institution's name and the email domain used to verify students. A university has many faculties, and every user belongs to one university.

Faculty

Represents a faculty inside a university (e.g. Engineering, Built Environment and IT). Stores the faculty name plus a foreign key back to the university it belongs to. A faculty offers many modules, sitting between University and Module in the hierarchy.

User

Represents a registered user of the platform. Stores identity info (name, email, password hash), verification status, role (student or admin), and faculty. A user can have many listings (as a seller), many audit log entries (as an admin who performed moderation actions), many notifications, many reports (as the reporter), and many appeal cases (as

the banned party). A user's university_id gets set to NULL if the university record is deleted (ON DELETE SET NULL), so the account itself isn't lost. New for Demo 3: `is_banned`, `banned_at`, `banned_by` (admin UUID), and `ban_reason` record whether the account is currently banned (UC10). These are columns on the user record itself, not a separate ban table; the historical record of *why* and *when* a ban happened lives in AuditLog.

OTP

Represents a one-time password record created during registration. Stores the target email, the 6-digit code, the expiry timestamp, a used flag, and when it was created. Each OTP can only be used once.

Module

Represents a university module (course). Stores the module code (e.g. COS301), module name, and semester, plus a foreign key to the faculty that offers it. A module can be linked to many listings.

Book

Represents the actual physical book being listed. Stores ISBN, title, author, edition, and publisher. The same book can appear in multiple listings (e.g. the same textbook listed by different sellers).

Listing

The central entity of the whole platform. Represents one student's offer to sell a specific book. A listing links a seller (User), a book (Book), and a module (Module). It stores price, condition, annotation level, photo_urls, and a status. As of Demo 3 the status enum is `PENDING`, `APPROVED`, `REJECTED`, `RESERVED`, `SOLD`, and `SOFT_DELETED` (UC5.2 added `RESERVED` and `SOLD` as an intermediate lifecycle between `APPROVED` and the listing leaving circulation). It also stores reviewed_by (admin UUID), reviewed_at, deleted_at, and created_at, so there's a full record of its lifecycle. A listing can also have many reports made against it (UC10) and can appear on many students' wishlists.

AuditLog

Records every moderation and account-standing action an admin performs. Stores entity_type, entity_id (the listing or user UUID), performed_by (admin UUID), performed_at, and optional notes. As of Demo 3, `action` covers `APPROVED`, `REJECTED`, and `BANNED` (UC10); an appeal Case's `ban_id` points back at the specific `BANNED` entry it's appealing. Basically gives us an immutable accountability trail.

SavedSearch

Represents a filter combination a student has chosen to save for later. Stores a foreign key to the owning User, a `filter_json` field holding the saved filter parameters, and a

creation timestamp. A user can have many saved searches.

Wishlist
A join entity linking a User to a Listing they've saved. Uses a composite primary key of `user_id` and `listings_id`, both with `ON DELETE CASCADE`, so a wishlist entry gets removed automatically if either the user or the listing gets deleted.

Notification (new for Demo 3, UC9)
Represents a single in-app notification generated by `NotificationService.emit()` for one user. Stores a foreign key to the recipient User, a `type` (e.g. `LISTING_APPROVED`, `SMART_FILTER_MATCH`, `LISTING_NEEDS_REVIEW`), a `payload` JSON blob with the details needed to render it, an `is_read` flag, and a creation timestamp. There is deliberately no `link_url` column; the frontend resolves where a notification should navigate to from its `type` using a client-side lookup map, so routes can be restructured later without a data migration. A user can have many notifications.

Report (new for Demo 3, UC10)
Represents a student's report against a listing. Stores a foreign key to the reporting User, a foreign key to the reported Listing, a `reason`, a `status` (`PENDING` or `REVIEWED`), and a creation timestamp. A user can submit many reports, and a listing can receive many reports.

Case (new for Demo 3, UC10)
Represents a banned user's appeal against their ban. Stores a foreign key to the banned User, a foreign key (`ban_id`) back to the specific `BANNED` AuditLog entry being appealed, an `appeal_message`, a `status` (`PENDING`, or `REVERSED`), a foreign key to the reviewing admin User (`reviewed_by`), and `reviewed_at`. A user can have many cases over time (one per ban), and an admin can review many cases.

Auction (new for Demo 4, UC12)
One-to-one with an existing `APPROVED` Listing. Holds the auction's current, fast-read state, used both to render the auction page and as the row locked by `SELECT ... FOR UPDATE` during bid processing. Stores `listing_id` (foreign key, unique), `current_price`, `reserve_price`, `end_time`, `extension_count`, and `status` (`ACTIVE` or `CLOSED`). The underlying listing continues to use its existing status lifecycle (UC5.2) once the auction closes and a winner is determined.

Bid (new for Demo 4, UC12)
An append-only audit log of every bid placed on an auction, including rejected bids with a reason; rows are never updated or deleted. Stores a foreign key to the Auction, a foreign key to the bidding User, an `amount`, a `status` (`ACCEPTED` or `REJECTED`), an optional `rejection_reason`, and a `created_at` timestamp. An auction can have many bids, and

a user can place many bids across many auctions.

7. Appendix

This section keeps earlier versions of SRS sections that have since changed. Sections get moved here once they're substantially updated, rather than just deleted, so there's still a record of how our requirements evolved.

7.1 Previous Versions

Version 1.0 (Sprint 1)

The very first version of this document, with just the Introduction and User Stories. Moved here after Sprint 2, once we added use cases, functional requirements, API contracts, the domain model, and architectural requirements.

Version 2.0 Functional Requirements (pre-realignment)

The original Sprint 2 draft numbered functional requirements with flat, service-prefixed IDs (e.g. `FR-AUTH-01`, `FR-LIST-02`, `FR-MOD-03`, `FR-MOD-S01`). We've since replaced this with the hierarchical FR1 / FR1.1 / FR1.1.1 numbering shown in Section 4, which matches the requirement breakdown format from the Systems Engineering lectures (Ch. 3–4).

Sprint 2 Closure Notes (18 May 2026)

- Three use cases signed off: UC1 (Create listing), UC2 (View listing), UC3 (Review listings).
- Registration and login are base features and don't count toward the three use cases.
- Authentication method: OTP sent to university email.
- Email allowlist: `@tuks.co.za` and `@up.ac.za` only.
- Module code search-as-you-type lookup, filterable by university.
- Listing lifecycle uses soft delete with an audit log.
- Branding: follows the Agile Bridge website (`www2.agilebridge.co.za`).
- Renting feature: stubbed out for now, there are legal considerations we haven't figured out.

Demo 2 Closure Notes (30 July 2026)

- Five new use cases delivered: UC4 (Messaging), UC5 (Edit/Withdraw Listing), UC6 (Smart Filters and Saved Searches), UC7 (Wishlist), UC8 (Audit Log Query).

- Architectural requirements, technology requirements, and API contracts have been moved out of this document into the separate SAS document, as required for Demo 2.
- Added Non-Functional Requirements as its own section, with five quantified quality attributes.
- Fixed the email allowlist throughout this document. There are actually three supported domains, `@tuks.co.za` (the main one), `@uct.ac.za`, and `@wits.ac.za`. `@up.ac.za` was never actually a real supported domain and has been removed everywhere it used to appear.
- The notification-core system (event bus, notifications table, bell icon UI), which was originally scoped for Demo 2, has been moved to Demo 3.

Demo 3 Closure Notes (31 August 2026)

- Five new use cases delivered: UC5.2 (Reserved and Sold Listing Status), UC9 (Notification System), UC9.1 (Smart Filter Match Alerts), UC9.2 (Listing Lifecycle Notifications), and UC10 (Report, Ban and Appeal Management).
- Added corresponding functional requirements: R6.2 (Reserved and Sold Status, extending the ListingService), R10 (Notifications, covering UC9, UC9.1, and UC9.2), and R11 (Reports, Bans and Appeals, extending the ModerationService).
- Added user stories US-B06 to US-B13, US-S05 to US-S10, and US-A06 to US-A09, giving every new use case (and several previously-undocumented parts of existing ones, such as messaging and filtering from the buyer's side) an explicit user story.
- Rewrote the Non-Functional Requirements section so every quality attribute is backed by a specific number already true of the deployed system (bcrypt cost factor, rate limits, token lifetimes, replica counts, measured Lighthouse scores, and so on) rather than an unquantified aspiration, and added a new Auditability requirement tied to UC10.
- Extended Definitions and Abbreviations with the new use cases and the vocabulary UC5.2/UC9/UC10 introduce: `RESERVED`, `SOLD`, `BANNED`, `REVERSED`, and `emit()`.
- The Domain Model (Section 8) has been extended with three new entities – Notification, Report, and Case – and updated `User` (ban fields) and `Listing` (extended status enum) descriptions. `Case.user_id` is modelled as a foreign key into AuditLog rather than a separate Bans table, since no such table exists elsewhere in the schema; this should be confirmed against the actual migration.

Demo 4 Documentation Update (25 September 2026, in progress)

This entry documents this revision of the document, made while Sprint 6 is still underway rather than after the fact; UC12 and UC13 are still being finished as of this update, so some detail below (NFR evidence, the domain-model diagram graphic) is flagged as pending rather than claimed as complete.

- Three new wow-factor use cases added: UC11 (AI-Assisted Listing Creation, extending UC1), UC12 (Real-Time Textbook Auctions), and UC13 (Smart Multi-Book Bundle Optimizer).
- Added corresponding functional requirements: R12 (VisionService, UC11), R13 (AuctionService, UC12), and R14 (BundleService, UC13).
- Added user stories US-S11, US-S12, US-B14, and US-B15.
- Added a Concurrency row to Non-Functional Requirements, tied to UC12's `SELECT ... FOR UPDATE` bid-locking guarantee, and extended the Performance row with quantified targets for `POST /vision/extract-text` and `POST /bundles/optimize`. Evidence for these two new targets is still being collected as part of this sprint's NFR testing pass and is not yet captured in this document.
- Added the missing "Delivered for Demo 3" entry to Scope (UC5.2, UC9, UC9.1, UC9.2, UC10 were already functional requirements elsewhere in this document but had never actually been listed there), and added a "Delivered for Demo 4" entry alongside it.
- Extended Definitions and Abbreviations with UC11–UC13 and the vocabulary they introduce: `OCR`, Reserve Price, and Anti-sniping.
- The Domain Model (Section 8) has been extended with two new entities – Auction and Bid.
- Expanded the email domain allowlist (R1.4, the Constraints table, UC0, and the "Not in scope" list) from five supported universities to all 26 South African public universities. While verifying the new entries against each university's own student IT documentation, two domains already in the allowlist turned out to be wrong: UCT was listed as `@uct.ac.za` (that's the staff/general domain; students actually use `@myuct.ac.za`) and Wits was listed as `@wits.ac.za` (students actually use `@students.wits.ac.za`). Both are corrected in R1.4's table. Only 10 of the 26 domains are confirmed this way; the remaining 16 are an unverified guess at the general institutional domain and are explicitly flagged as such rather than presented with false confidence, since the pattern in the 10 confirmed entries suggests a guess at the general domain is more likely wrong than right. The backend `.env` allowlist

and the SAS Constraints section have not been synced to this yet; that is tracked as a follow-up, not assumed done.

- **Superseded in version 5.2 (28 September 2026):** the allowlist now contains confirmed student domains only, and the 16 unverified guesses from the previous item have been removed. While re-checking them, five turned out to be wrong for students (UJ, UNISA, UWC, CPUT and DUT each run student email on a different domain from the general one), so those five were corrected and confirmed. The result is 15 universities on the allowlist and 11 omitted until their student domain is confirmed. See R1.4.

Uni Textbook Marketplace

In collaboration **with:**

agilebridge
CONSIDERED
SOFTWARE SOLUTIONS

NEXUSDEV

2026 NexusDev - University of Pretoria - COS 301 Capstone Project
This document is version-controlled in the GitHub repository under `/docs/Demo 4/Software Requirements Specifications.pdf`